

Microservices Architecture — The Complete Guide

Monolith vs Microservices • Spring Boot • Java Full Stack

Real-Time Banking Example: Accounts, Cards & Loans Services

Service Discovery, API Gateway, Resilience, Messaging, Security, Observability, Cloud & Kubernetes

A practical, simple-language, end-to-end reference for interview preparation and real-world implementation

Table of Contents

1. Types of Applications — Frontend & Backend Systems Overview
2. Monolithic Architecture Explained (with Diagram)
3. Microservices Architecture Explained (with Diagram)
4. Monolith vs Microservices — Full Comparison
5. Advantages & Disadvantages of Monolithic Architecture
6. Advantages & Disadvantages of Microservices Architecture
7. When to Use Monolith vs Microservices — Decision Guide
8. Real-World Example — Banking App: Accounts, Cards & Loans (3 Microservices)
9. Migrating from Monolith to Microservices (Strangler Fig Pattern)
10. Setting Up a Real Spring Boot Microservices Project (Multi-Module)
11. Service Discovery — Netflix Eureka
12. API Gateway — Spring Cloud Gateway
13. Inter-Service Communication — REST
14. Feign Client — Declarative REST Calls
15. gRPC Basics for Microservices
16. Load Balancing (Client-Side & Server-Side)
17. Distributed Configuration — Spring Cloud Config Server
18. Circuit Breaker Pattern — Resilience4j
19. Retry Pattern
20. Bulkhead Pattern
21. Rate Limiting
22. Distributed Transactions & the Saga Pattern
23. Choreography vs Orchestration
24. Event-Driven Architecture
25. Synchronous vs Asynchronous Communication
26. Idempotency in Microservices
27. Database per Service Pattern
28. CQRS — Basic Understanding
29. API Versioning Strategies
30. Centralized Logging — ELK Stack / Splunk
31. Distributed Tracing — OpenTelemetry, Zipkin, Jaeger
32. Monitoring & Metrics — Prometheus & Grafana
33. Service Registry, Health Checks & Spring Boot Actuator
34. Security in Microservices — JWT, OAuth2, Keycloak
35. Caching — Redis
36. Message Brokers — Kafka & RabbitMQ
37. Containerization (Docker) & Orchestration (Kubernetes) Recap for Microservices
38. Deployment Strategies — Blue-Green, Canary, Rolling
39. Fault Tolerance & High Availability

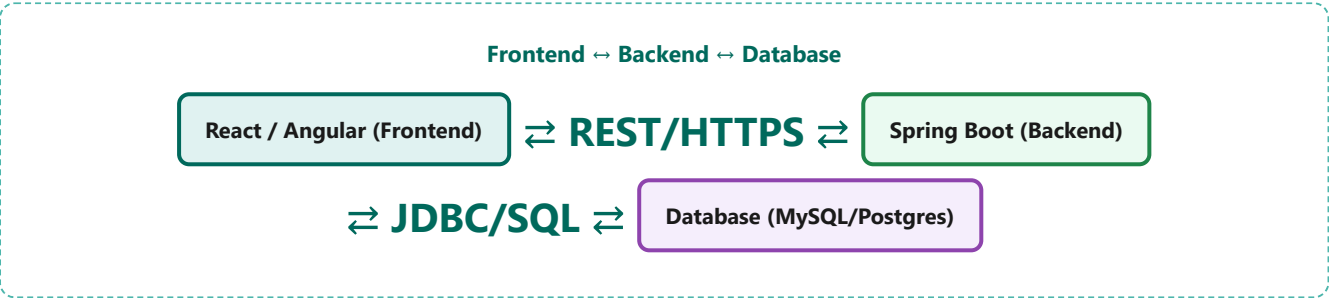
- 40. How We Implement Microservices in Real-Time (End-to-End)
- 41. How to Maintain Microservices in Production (Real Organization Practices)
- 42. Cloud Deployment — Azure Spring Apps, AKS and Azure Integration
- 43. Common Microservices Issues, Problems & Solutions
- 44. Interview Questions & Answers (Basic → Advanced → Scenario Based)
- 45. Final Summary & Cheat Sheet

1. Types of Applications — Frontend & Backend Systems

Overview

1.1 Frontend vs Backend — Simple Terms

Frontend is everything the user directly sees and interacts with (React, Angular, mobile apps). **Backend** is everything running on the server that the frontend talks to — business logic, database access, security, integrations (Spring Boot, Node.js, .NET).



1.2 Types of Backend Application Architectures

Architecture Type	Description	Typical Technology
Monolithic	One large codebase/deployable unit containing all business logic	Single Spring Boot WAR/JAR
Microservices	Application split into many small, independently deployable services	Multiple Spring Boot apps + Eureka + Gateway
Serverless	Code runs as short-lived functions triggered by events, no server management	Azure Functions, AWS Lambda
Service-Oriented Architecture (SOA)	Predecessor to microservices — larger, coarser-grained services, often with an Enterprise Service Bus (ESB)	SOAP/XML, legacy enterprise systems

1.3 Where This Guide Fits

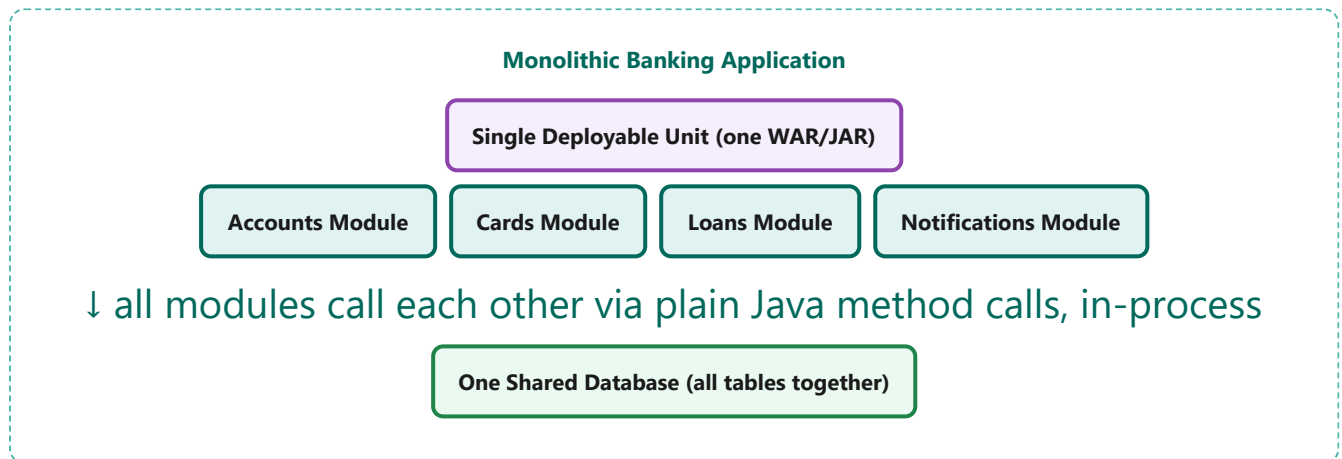
This guide focuses on **Monolithic vs Microservices** architecture for Java/Spring Boot backend systems (with React as a typical frontend), covering the full real-world toolset organizations use to build, communicate between, secure, monitor, and deploy microservices — using a real banking-style example (Accounts, Cards, Loans).

2. Monolithic Architecture Explained (with Diagram)

2.1 What Is a Monolith?

A monolithic application is built and deployed as **one single unit** — all modules (accounts, cards, loans, notifications, reporting) live in one codebase, share one database, compile into one WAR/JAR, and are deployed together on one (or a few identical) application server instances.

2.2 Monolith Diagram — Banking Example



2.3 Internal Structure of a Typical Monolith

```
bank-app/
├─ src/main/java/com/bank/
│  ├─ accounts/  (controller, service, repository, entity)
│  ├─ cards/     (controller, service, repository, entity)
│  ├─ loans/     (controller, service, repository, entity)
│  └─ notifications/
├─ src/main/resources/application.yml
└─ pom.xml → builds ONE bank-app.war
```

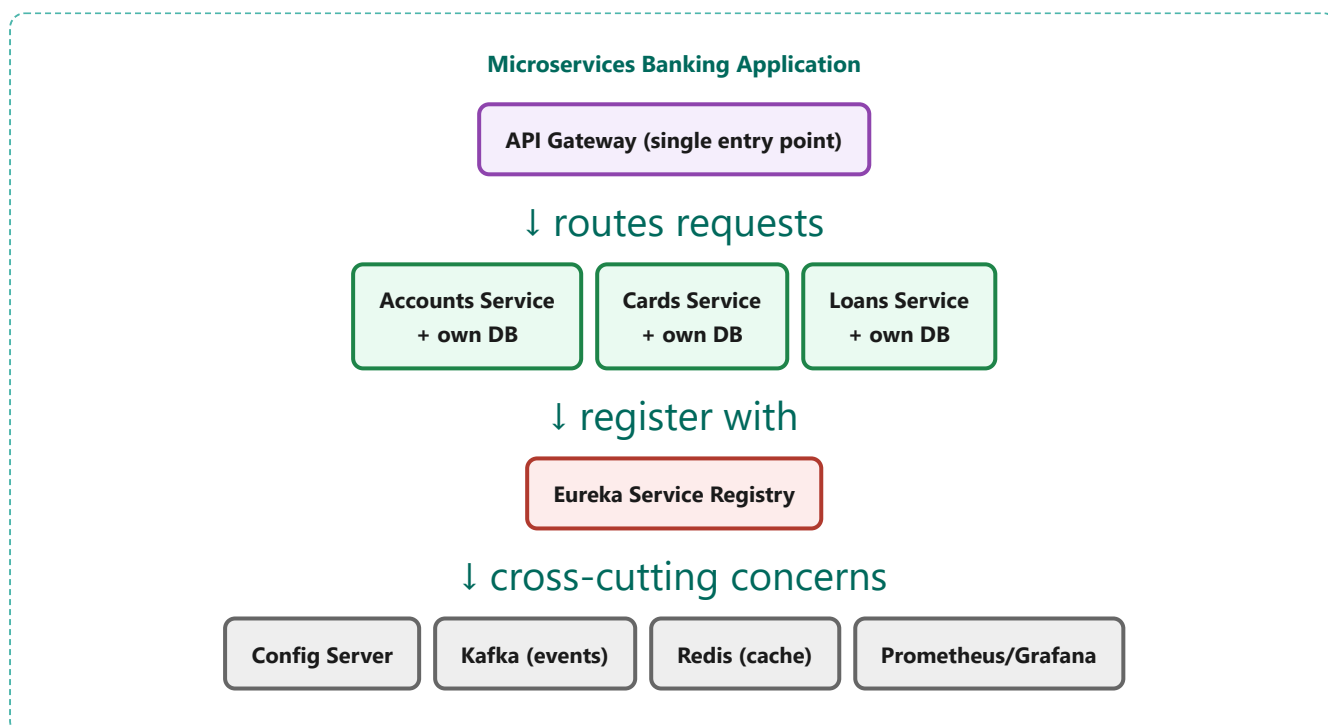
Notice modules can still be organized cleanly by package/layer — the defining trait of a monolith isn't "messy code," it's that everything is **compiled, deployed, scaled, and released together as one unit**.

3. Microservices Architecture Explained (with Diagram)

3.1 What Are Microservices?

Microservices architecture splits an application into multiple small, independently deployable services, each owning its own business capability, its own codebase, its own database, and its own deployment lifecycle — services communicate over the network (REST, gRPC, messaging) instead of in-process method calls.

3.2 Microservices Diagram — Same Banking Example



3.3 Key Characteristics of Microservices

- **Independently deployable** — deploying Cards Service doesn't require redeploying Accounts or Loans.
- **Own data store** — each service owns its schema/database, no other service touches it directly (Database per Service pattern — Section 27).
- **Communicate over the network** — REST, gRPC, or asynchronous messaging (Kafka/RabbitMQ), never direct in-process calls.
- **Organized around business capability** — "Accounts," "Cards," "Loans" are business domains, not technical layers.
- **Decentralized governance** — different services can (in theory) use different languages/databases, though most Java shops keep it consistent with Spring Boot across services.
- **Built for failure** — resilience patterns (circuit breaker, retry, bulkhead) assume any service call might fail.

4. Monolith vs Microservices — Full Comparison

Aspect	Monolith	Microservices
Codebase	Single repository/codebase	Multiple repositories, one per service (typical)
Deployment	One deployable unit, all-or-nothing release	Independent deployment per service
Database	Usually one shared database	One database per service
Scaling	Scale the entire app, even if only one module is under load	Scale only the service under load (e.g., just Cards Service during a promo)
Technology choice	Single tech stack for the whole app	Each service can pick its own stack (polyglot), though often standardized
Team structure	One large team, or several teams stepping on each other's code	Small autonomous teams, one (or few) per service ("you build it, you run it")
Communication	In-process Java method calls — fast, simple	Network calls (REST/gRPC/messaging) — slower, needs resilience handling
Testing	Simpler end-to-end testing (all in one process)	Complex — needs contract testing, service virtualization, integration environments
Failure isolation	One bug can crash the entire application	A failing service can be isolated (with circuit breakers) without taking down the whole system
Operational complexity	Low — one thing to monitor/deploy	High — needs service discovery, distributed tracing, centralized logging, API gateway
Release speed	Slower — any change requires full regression + full redeploy	Faster per-service — teams release independently, more frequently
Initial development speed	Faster to start — no distributed system complexity upfront	Slower to start — infrastructure (registry, gateway, config server) must exist first

5. Advantages & Disadvantages of Monolithic Architecture

5.1 Advantages

- Simple to develop, test, and deploy initially — one build, one deployable artifact.
- Easier debugging — a single stack trace across the whole request, no network hops to trace.
- No distributed system complexity — no service discovery, no network latency between modules, no partial failure handling.
- Simpler transactions — a single database means ACID transactions across modules are trivial.
- Lower operational overhead — one thing to monitor, one thing to deploy, one log file.
- Good fit for small teams and early-stage products/startups where speed of initial delivery matters more than independent scaling.

5.2 Disadvantages

- Scaling is all-or-nothing — you can't scale just the "Cards" module without scaling the entire app.
- Slower release cycles — a tiny change in one module requires rebuilding/retesting/redeploying everything.
- Technology lock-in — hard to adopt a new framework/language for just one part of the system.
- Team bottleneck — many developers committing to the same codebase increases merge conflicts and coordination overhead as the team grows.
- Larger blast radius for bugs — a memory leak or bug in one module can crash the entire application, including unrelated modules.
- Harder to onboard new developers as the codebase grows very large over years.

6. Advantages & Disadvantages of Microservices Architecture

6.1 Advantages

- Independent scaling — scale only the service experiencing load (e.g., Loans Service during month-end processing).
- Independent deployment — release Cards Service ten times a day without touching Accounts or Loans.
- Fault isolation — with circuit breakers, a failing Loans Service doesn't necessarily take down Accounts or Cards.
- Technology flexibility — teams can choose the best database/framework per service's needs.
- Team autonomy — small teams own a service end-to-end ("you build it, you run it"), improving ownership and velocity.
- Easier to understand a single service in isolation, once you know the boundaries.

6.2 Disadvantages

- Distributed system complexity — network latency, partial failures, eventual consistency all become real concerns.
- Operational overhead — needs service discovery, API gateway, centralized logging, distributed tracing, container orchestration.
- Data consistency is hard — no more simple cross-module ACID transactions; requires Sagas/eventual consistency (Section 22).
- Testing is harder — end-to-end tests span multiple services/network calls; needs contract testing and test environments per integration.
- Debugging is harder — a single user request might touch 5 services; requires distributed tracing to follow the full path.
- Higher infrastructure cost — many small JVMs/containers running 24/7 vs one larger one, more moving parts to secure and patch.
- Requires DevOps maturity — CI/CD, containerization, and monitoring must all be solid before microservices pay off.

Interview tip: Microservices are not "free" — they trade development simplicity for operational flexibility and independent scaling. An organization without strong CI/CD, monitoring, and DevOps practices will often struggle more with microservices than they would with a well-organized monolith.

7. When to Use Monolith vs Microservices — Decision Guide

Situation	Recommended Approach	Why
Early-stage startup, small team, unclear requirements	Monolith (well-modularized)	Fast iteration, no premature distributed system complexity, requirements still shifting
Large enterprise with multiple independent teams	Microservices	Teams need to release independently without blocking each other
Specific module has very different scaling needs (e.g., payments spike at month-end)	Microservices (at least for that module)	Scale only what needs scaling, save infrastructure cost elsewhere
Strict regulatory/compliance boundaries between business domains (e.g., loans vs accounts)	Microservices	Clean data/access boundary per domain, easier to audit and secure independently
Small internal tool, low traffic, one team	Monolith	Operational overhead of microservices isn't justified
Organization lacks strong CI/CD, containerization, monitoring maturity	Monolith first	Build DevOps maturity before taking on distributed system complexity
Need to release different parts of the system at very different velocities	Microservices	Decouples release cadence per team/service

Common real-world pattern: "Start with a well-modularized monolith (modular monolith), and split into microservices only when a specific module has a proven, real need for independent scaling/release/team ownership." Many organizations over-engineer by starting with microservices on day one, before they even understand their domain boundaries.

8. Real-World Example — Banking App: Accounts, Cards & Loans (3 Microservices)

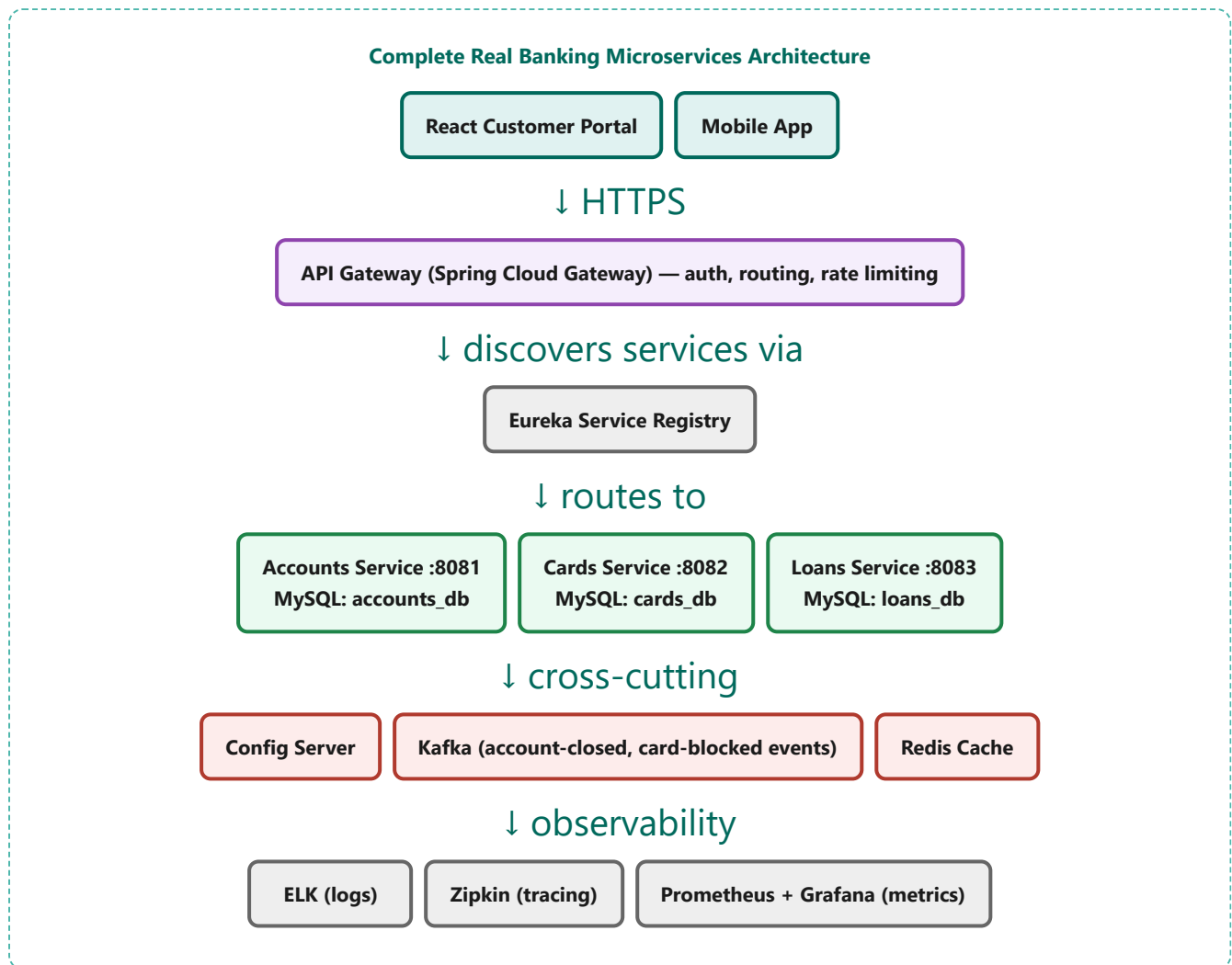
8.1 The Scenario

Our organization's core banking platform is split into three microservices — **Accounts Service**, **Cards Service**, and **Loans Service** — each owned by a different team, each with its own Spring Boot codebase, own database schema, and own release cycle. A React customer portal and a mobile app both talk to these services through a single API Gateway.

8.2 Service Responsibilities

Service	Responsibility	Own Database Tables
Accounts Service	Customer accounts, balances, transactions, statements	customer, account, transaction
Cards Service	Debit/credit card issuance, limits, blocking/unblocking, card transactions	card, card_limit, card_transaction
Loans Service	Loan applications, approval workflow, EMI schedules, repayments	loan, loan_schedule, repayment

8.3 Full Architecture Diagram



8.4 A Real Cross-Service Flow — "Close Account" Use Case

1. Customer calls `DELETE /accounts/{id}` via the Gateway → routed to Accounts Service.
2. Accounts Service checks: does this account have any active cards or open loans? It calls Cards Service and Loans Service (via Feign/REST) to confirm.
3. If clear, Accounts Service marks the account closed and publishes an `AccountClosedEvent` to Kafka.
4. Cards Service (Kafka consumer) automatically deactivates any lingering cards linked to that account.
5. Loans Service (Kafka consumer) flags the customer's loan record as "account closed" for compliance reporting.

Why this matters: This single example touches synchronous REST calls (immediate validation), asynchronous events (Kafka, for eventual side effects), and shows why each service having its **own** database means cross-service checks must go through APIs/events — never direct SQL joins across services.

9. Migrating from Monolith to Microservices (Strangler Fig Pattern)

9.1 Why Not a "Big Bang" Rewrite?

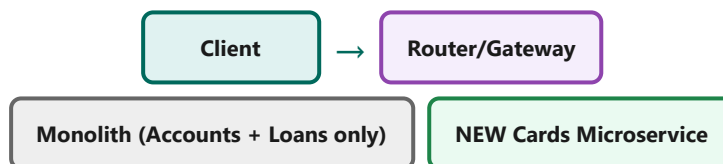
Rewriting an entire monolith into microservices all at once is extremely risky — it freezes feature delivery for months/years, and by the time it's done, business requirements have often changed. Real organizations use the **Strangler Fig Pattern**: gradually carve out and replace pieces of the monolith while it keeps running in production.

9.2 Strangler Fig Pattern — Step by Step

Strangler Fig Migration — Step 1

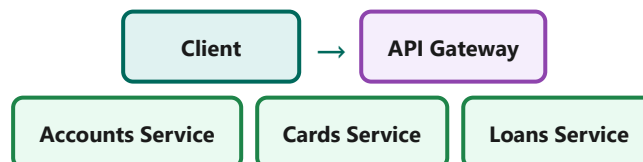


Strangler Fig Migration — Step 2 (Extract Cards first)



The Gateway routes `/cards/**` traffic to the new Cards Service, while everything else still goes to the monolith. The monolith's own Cards module is eventually deleted once the new service is fully verified in production.

Strangler Fig Migration — Final State



The original monolith no longer exists — it has been fully "strangled" and replaced, one module at a time, with zero big-bang cutover risk.

9.3 Practical Migration Steps

1. **Identify bounded contexts** using Domain-Driven Design — figure out natural seams (Accounts, Cards, Loans are separate business domains).

2. **Pick the least risky/most isolated module first** (often Cards, since it has fewer cross-cutting dependencies than Accounts).
3. **Extract the module's database tables** into a new schema/database dedicated to the new service.
4. **Build the new Spring Boot microservice**, replicate the module's functionality, add integration tests comparing old vs new behavior.
5. **Introduce an API Gateway/router** in front of both monolith and new service, routing by path.
6. **Dual-write or sync data** temporarily if needed during transition (e.g., CDC — Change Data Capture — from old DB to new DB).
7. **Cut over traffic gradually** (Canary-style — 5% → 50% → 100%) to the new service, monitoring error rates closely.
8. **Remove the old module's code and tables** from the monolith once fully migrated and stable.
9. **Repeat** for the next module (Loans, then Accounts) until the monolith is fully replaced.

Real scenario: "Our bank ran the old monolith and new Cards microservice side by side for six weeks, with the Gateway splitting traffic by customer ID hash — this let us validate the new service's correctness on real production traffic without risking a full cutover, and roll back instantly by just changing the routing rule if issues appeared."

10. Setting Up a Real Spring Boot Microservices Project (Multi-Module)

10.1 Real Repository/Module Layout

```
bank-microservices/  
├─ accounts-service/      (Spring Boot app, port 8081)  
├─ cards-service/         (Spring Boot app, port 8082)  
├─ loans-service/         (Spring Boot app, port 8083)  
├─ eureka-server/         (Spring Boot app, port 8761)  
├─ config-server/         (Spring Boot app, port 8888)  
├─ api-gateway/           (Spring Boot app, port 8072)  
└─ common-lib/            (shared DTOs, exception classes – published as a small internal library)
```

Each service is a **separate Spring Boot project** with its own `pom.xml`, its own Git repository (or a mono-repo folder with independent CI pipelines), and its own database connection — deployed and scaled completely independently.

10.2 Core Spring Cloud Dependencies Used Across This Guide

```
<properties>  
  <spring-cloud.version>2023.0.3</spring-cloud.version>  
</properties>  
<dependencyManagement>  
  <dependencies>  
    <dependency>  
      <groupId>org.springframework.cloud</groupId>  
      <artifactId>spring-cloud-dependencies</artifactId>  
      <version>${spring-cloud.version}</version>  
      <type>pom</type>  
      <scope>import</scope>  
    </dependency>  
  </dependencies>  
</dependencyManagement>
```

10.3 Typical Accounts Service pom.xml Dependencies

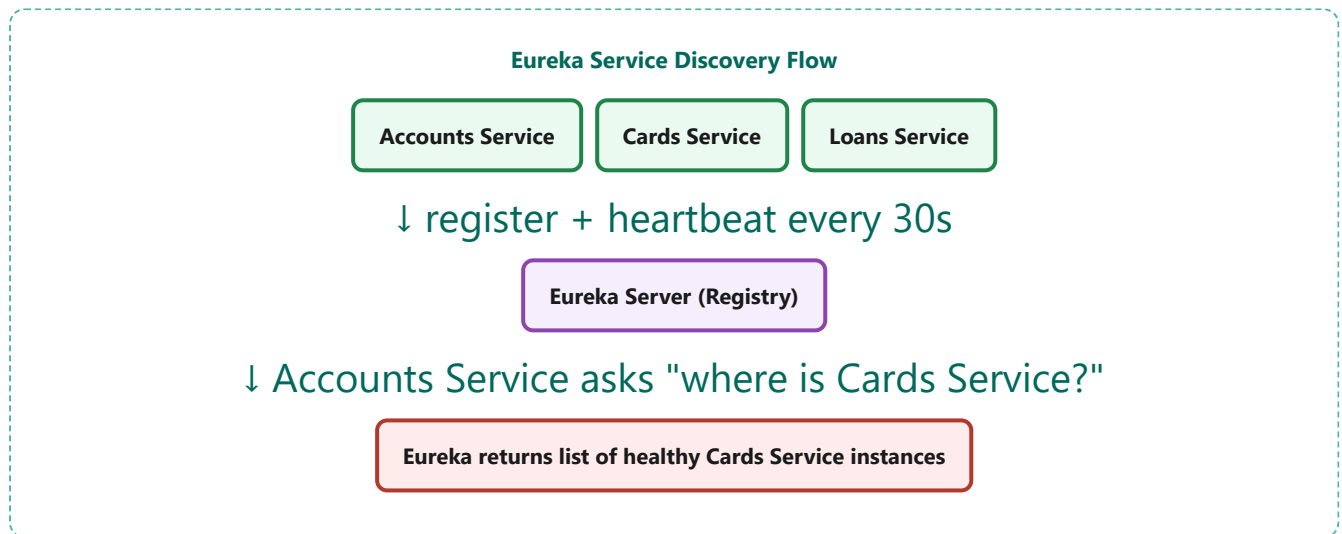
```
<dependencies>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</artifactId>
</dependency>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-
jpa</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
actuator</artifactId></dependency>
  <dependency><groupId>org.springframework.cloud</groupId><artifactId>spring-cloud-starter-netflix-eureka-
client</artifactId></dependency>
  <dependency><groupId>org.springframework.cloud</groupId><artifactId>spring-cloud-starter-
config</artifactId></dependency>
  <dependency><groupId>org.springframework.cloud</groupId><artifactId>spring-cloud-starter-circuitbreaker-
resilience4j</artifactId></dependency>
  <dependency><groupId>org.springframework.cloud</groupId><artifactId>spring-cloud-starter-
openfeign</artifactId></dependency>
  <dependency><groupId>mysql</groupId><artifactId>mysql-connector-j</artifactId></dependency>
</dependencies>
```

11. Service Discovery — Netflix Eureka

11.1 The Problem Service Discovery Solves

In a dynamic environment (containers restarting, scaling up/down, IPs changing constantly), hardcoding "Cards Service is at 10.0.4.23:8082" breaks the moment that pod restarts with a new IP. Service Discovery lets services register themselves by **name** and find each other dynamically at runtime.

11.2 How Eureka Works



11.3 Eureka Server Setup

```
// EurekaServerApplication.java
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}

# application.yml
server.port: 8761
eureka:
  client:
    register-with-eureka: false # the server itself doesn't need to register
    fetch-registry: false
```

11.4 Eureka Client Setup (Each Microservice)

```
@SpringBootApplication
@EnableDiscoveryClient
public class AccountsServiceApplication { ... }

# application.yml
spring.application.name: accounts-service
eureka:
  client:
    service-url:
      defaultZone: http://eureka-server:8761/eureka/
  instance:
    prefer-ip-address: true
```

Real scenario: "When Cards Service scales from 2 to 6 pods during a promotional campaign, Eureka's registry automatically reflects all 6 healthy instances within seconds — Accounts Service's Feign client picks a healthy instance via client-side load balancing without any manual configuration change."

12. API Gateway — Spring Cloud Gateway

12.1 Why an API Gateway?

Without a gateway, the frontend/mobile app would need to know the address of every microservice individually, handle auth in multiple places, and every service would need to implement rate limiting/CORS separately. The API Gateway is a single entry point that handles routing, authentication, rate limiting, and cross-cutting concerns once, centrally.

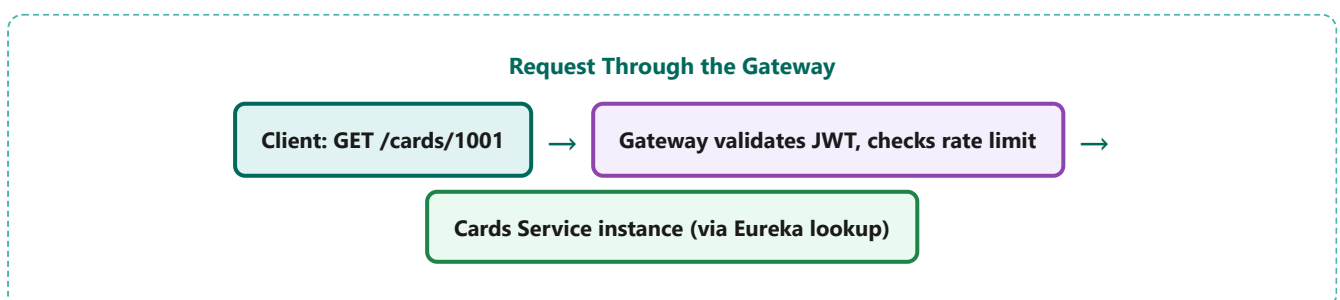
12.2 Gateway Responsibilities

- **Routing** — forwards `/accounts/**` to Accounts Service, `/cards/**` to Cards Service, etc.
- **Authentication/Authorization** — validates JWT tokens once at the edge (Section 33).
- **Rate limiting** — protects backend services from being overwhelmed (Section 20).
- **Load balancing** — spreads requests across healthy service instances (via Eureka integration).
- **Cross-cutting concerns** — CORS, request logging, response header injection.

12.3 Real Gateway Configuration

```
# application.yml - api-gateway
spring:
  cloud:
    gateway:
      routes:
        - id: accounts-route
          uri: lb://ACCOUNTS-SERVICE      # lb:// = load-balanced via Eureka
          predicates:
            - Path=/accounts/**
          filters:
            - RewritePath=/accounts/(?<segment>.*), /${segment}
        - id: cards-route
          uri: lb://CARDS-SERVICE
          predicates:
            - Path=/cards/**
        - id: loans-route
          uri: lb://LOANS-SERVICE
          predicates:
            - Path=/loans/**
```

12.4 Diagram — Gateway in the Full Flow



13. Inter-Service Communication — REST

13.1 Simple Synchronous REST Call

The most common way microservices talk directly to each other — a plain HTTP call, typically JSON over REST, using Spring's `RestTemplate` (older, being phased out) or the newer `WebClient` (reactive, recommended for new code).

```
@Service
public class AccountsService {

    private final WebClient webClient;

    public CardsDto getCardsForCustomer(Long customerId) {
        return webClient.get()
            .uri("http://CARDS-SERVICE/cards/customer/{id}", customerId)
            .retrieve()
            .bodyToMono(CardsDto.class)
            .block(); // blocking call – simplest to reason about for typical CRUD services
    }
}
```

13.2 REST Pros & Cons for Inter-Service Calls

Pros	Cons
Simple, widely understood, easy to debug (readable JSON, standard HTTP tools)	Synchronous — caller waits, coupling availability of caller to callee
Works with any client (browser, Postman, curl)	More network overhead than binary protocols (gRPC)
Easy to version, document (OpenAPI/Swagger)	No built-in streaming (without extra effort)

14. Feign Client — Declarative REST Calls

14.1 Why Feign?

Feign lets you call another microservice as if it were a simple Java interface — no manual URL building, no manual JSON parsing. Spring Cloud OpenFeign integrates directly with Eureka for service discovery and load balancing.

14.2 Real Feign Client Example

```
@FeignClient(name = "cards-service") // "cards-service" resolved via Eureka automatically
public interface CardsFeignClient {

    @GetMapping("/cards/customer/{customerId}")
    ResponseEntity<CardsDto> getCardsByCustomer(@PathVariable Long customerId);
}

// Usage inside AccountsService
@Service
@RequiredArgsConstructor
public class AccountsService {
    private final CardsFeignClient cardsFeignClient;

    public boolean hasActiveCards(Long customerId) {
        ResponseEntity<CardsDto> response = cardsFeignClient.getCardsByCustomer(customerId);
        return response.getBody() != null && response.getBody().isActive();
    }
}

// Enable in the main application class
@SpringBootApplication
@EnableFeignClients
public class AccountsServiceApplication { ... }
```

Interview tip: Feign is built on top of the same HTTP mechanics as RestTemplate/WebClient — its value is purely developer ergonomics (declarative interfaces) plus seamless integration with Eureka load balancing and Resilience4j fallback annotations.

14.3 Feign with Resilience4j Fallback

```
@FeignClient(name = "cards-service", fallback = CardsFeignFallback.class)
public interface CardsFeignClient { ... }

@Component
public class CardsFeignFallback implements CardsFeignClient {
    @Override
    public ResponseEntity<CardsDto> getCardsByCustomer(Long customerId) {
        return ResponseEntity.ok(CardsDto.empty()); // graceful degradation when Cards Service is down
    }
}
```

15. gRPC Basics for Microservices

15.1 What Is gRPC?

gRPC is a high-performance RPC (Remote Procedure Call) framework using **Protocol Buffers** (binary format) instead of JSON, and HTTP/2 instead of HTTP/1.1 — significantly faster and more compact than REST for high-throughput internal service-to-service calls.

15.2 Simple .proto Definition

```
syntax = "proto3";

service LoansService {
  rpc GetLoanDetails (LoanRequest) returns (LoanResponse);
}

message LoanRequest {
  int64 loanId = 1;
}

message LoanResponse {
  int64 loanId = 1;
  double outstandingAmount = 2;
  string status = 3;
}
```

15.3 REST vs gRPC — When to Use Which

Aspect	REST (JSON/HTTP)	gRPC (Protobuf/HTTP2)
Performance	Slower (text-based, HTTP/1.1 typical)	Much faster (binary, HTTP/2 multiplexing)
Human readability	High — easy to inspect/debug	Low — binary, needs tooling to inspect
Browser support	Native	Needs grpc-web proxy for browsers
Best fit	Public APIs, frontend-facing endpoints, Gateway-exposed APIs	High-throughput internal service-to-service calls, streaming use cases
Real banking example	Gateway → Accounts Service (customer-facing)	Loans Service ↔ Risk-Scoring engine (high-volume internal calls)

Real-world note: Most Java shops use REST/Feign as the default for microservice communication and reserve gRPC for specific high-throughput internal paths (e.g., a fraud-scoring service called on every transaction) where the performance gain is worth the added tooling complexity.

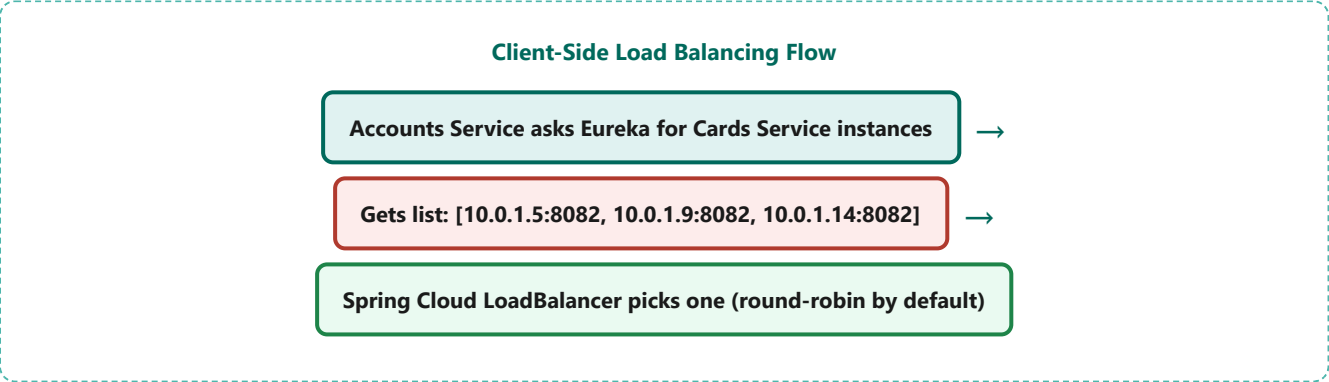
16. Load Balancing (Client-Side & Server-Side)

16.1 Server-Side Load Balancing

A dedicated load balancer (Azure Load Balancer, NGINX, Kubernetes Service) sits in front of service instances and distributes incoming requests — the caller doesn't know or care how many instances exist.

16.2 Client-Side Load Balancing (Spring Cloud LoadBalancer)

The calling service itself (via Eureka's list of healthy instances) picks which instance to call — used by Feign/WebClient with `lb://` URIs. This avoids needing a separate load balancer hop for internal service-to-service calls.



16.3 Load Balancing Strategies

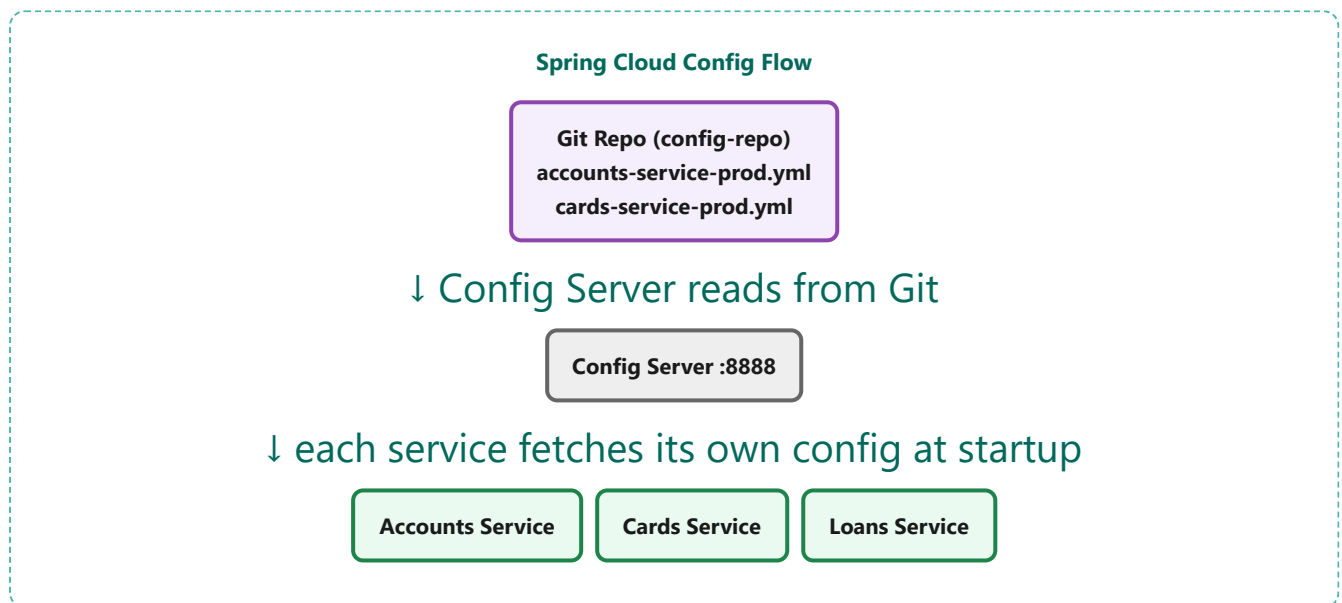
Strategy	How It Works
Round Robin	Cycles through instances in order — simple, default in many clients
Random	Picks a random healthy instance each time
Weighted Response Time	Favors instances that have historically responded faster
Least Connections	Sends traffic to the instance currently handling the fewest active requests

17. Distributed Configuration — Spring Cloud Config Server

17.1 The Problem It Solves

With dozens of microservices, each needing environment-specific config (DB URLs, feature flags, third-party API keys), managing individual `application.yml` files per service per environment becomes unmanageable. Config Server centralizes all configuration in one Git repository, served to every service at startup.

17.2 Architecture



17.3 Config Server Setup

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication { ... }

# application.yml
server.port: 8888
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/mybank/config-repo
          default-label: main
```

17.4 Client-Side Setup (Each Microservice)

```
# bootstrap.yml / application.yml in accounts-service
spring:
  application:
    name: accounts-service
  config:
    import: optional:configserver:http://config-server:8888
  profiles:
    active: prod
```

17.5 Refreshing Config Without Restart

```
@RefreshScope    // bean re-reads config when a /actuator/refresh event is triggered
@RestController
public class FeatureController {
    @Value("${feature.new-checkout-enabled}")
    private boolean newCheckoutEnabled;
}
```

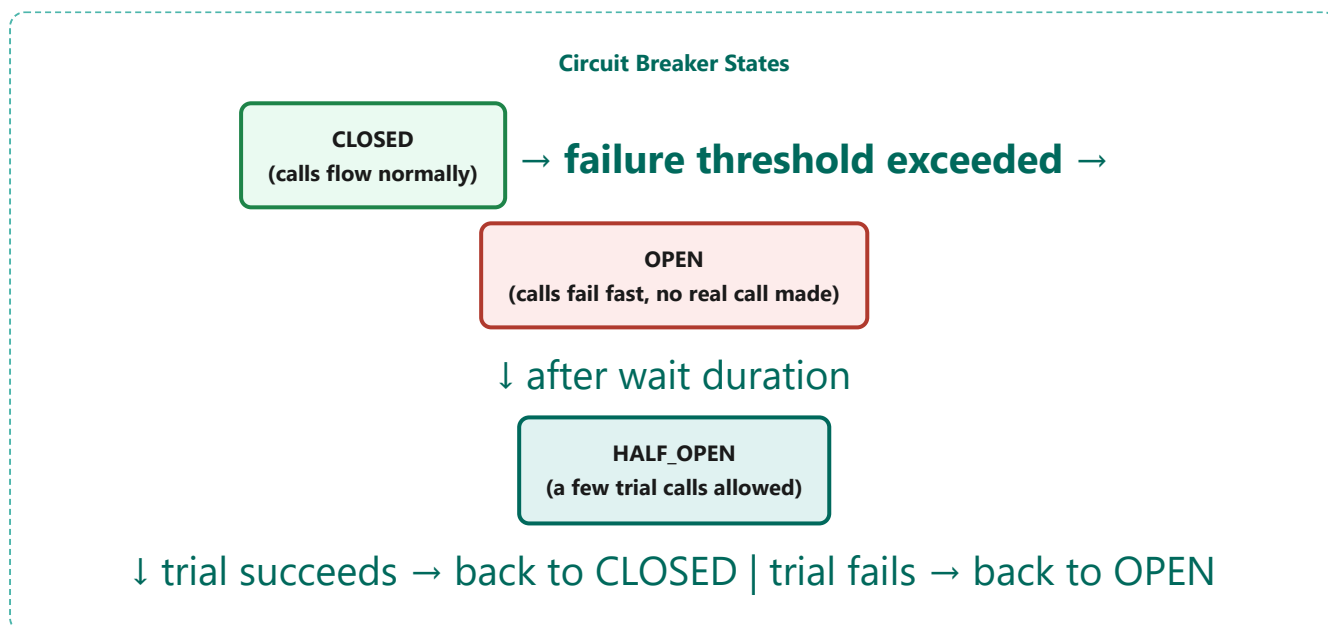
Real scenario: "During an incident, we needed to disable a risky new feature flag across all 12 running Accounts Service pods instantly — updating the value in the config Git repo and hitting `/actuator/refresh` (via Spring Cloud Bus + Kafka broadcasting the refresh event) applied the change everywhere within seconds, no redeploy needed."

18. Circuit Breaker Pattern — Resilience4j

18.1 The Problem — Cascading Failures

If Cards Service becomes slow/unresponsive, and Accounts Service keeps calling it and waiting on every request, Accounts Service's own threads get exhausted waiting — eventually Accounts Service becomes unresponsive too. One slow service can cascade into a total system outage.

18.2 How a Circuit Breaker Works



18.3 Resilience4j Circuit Breaker — Real Code

```
@Service
public class AccountsService {

    @CircuitBreaker(name = "cardsService", fallbackMethod = "cardsFallback")
    public CardsDto getCards(Long customerId) {
        return cardsFeignClient.getCardsByCustomer(customerId).getBody();
    }

    public CardsDto cardsFallback(Long customerId, Throwable t) {
        log.warn("Cards Service unavailable, returning fallback for customer {}", customerId);
        return CardsDto.empty(); // graceful degradation instead of full failure
    }
}

# application.yml
resilience4j:
  circuitbreaker:
    instances:
      cardsService:
        sliding-window-size: 10
        failure-rate-threshold: 50 # open circuit if 50% of last 10 calls failed
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 3
```

Interview tip: A Circuit Breaker doesn't prevent failures — it prevents a failing dependency from **cascading** into your own service by failing fast and giving the downstream service time to recover, while your own service stays responsive (even if in a degraded state).

19. Retry Pattern

19.1 When Retry Helps

Some failures are **transient** — a brief network blip, a momentary spike in load causing a timeout. Retrying the same call a few times, often with a growing delay (exponential backoff), can succeed without any real problem existing.

19.2 Resilience4j Retry — Real Code

```
@Retry(name = "cardsService", fallbackMethod = "cardsFallback")
@CircuitBreaker(name = "cardsService", fallbackMethod = "cardsFallback")
public CardsDto getCards(Long customerId) {
    return cardsFeignClient.getCardsByCustomer(customerId).getBody();
}

resilience4j:
  retry:
    instances:
      cardsService:
        max-attempts: 3
        wait-duration: 500ms
        exponential-backoff-multiplier: 2 # 500ms, 1000ms, 2000ms between attempts
        retry-exceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
```

19.3 Retry Danger — Non-Idempotent Operations

Retrying a "create payment" call blindly can create **duplicate payments** if the first attempt actually succeeded but the response was lost. Retry should only be applied to idempotent operations, or combined with an idempotency key (Section 25).

20. Bulkhead Pattern

20.1 The Ship Analogy

A ship is divided into watertight compartments ("bulkheads") — if one compartment floods, the rest of the ship stays afloat. The Bulkhead pattern applies the same idea to software: isolate resources (thread pools, connection pools) per dependency so one slow/failing dependency can't exhaust resources needed by unrelated calls.

20.2 Resilience4j Bulkhead — Real Code

```
@Bulkhead(name = "cardsService", type = Bulkhead.Type.THREADPOOL)
public CardsDto getCards(Long customerId) { ... }

resilience4j:
  thread-pool-bulkhead:
    instances:
      cardsService:
        max-thread-pool-size: 10
        core-thread-pool-size: 5
        queue-capacity: 20
```

Real scenario: "Before adding bulkheads, a slow Loans Service call was exhausting Accounts Service's entire shared HTTP thread pool, which also blocked unrelated Cards Service calls from getting a thread at all. Isolating each downstream call into its own bounded thread pool meant a slow Loans Service no longer starved Cards Service calls."

21. Rate Limiting

21.1 Why Rate Limit?

Protects services from being overwhelmed by too many requests (whether from a buggy client, a traffic spike, or an abusive/malicious client) — both at the API Gateway (protecting all backend services) and within individual services for specific expensive operations.

21.2 Rate Limiting at the Gateway (Redis-Backed)

```
spring:
  cloud:
    gateway:
      routes:
        - id: cards-route
          uri: lb://CARDS-SERVICE
          predicates:
            - Path=/cards/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 20    # 20 requests/sec allowed sustained
                redis-rate-limiter.burstCapacity: 40    # allow short bursts up to 40
```

21.3 Resilience4j Rate Limiter (Service-Level)

```
@RateLimiter(name = "loanApplication", fallbackMethod = "rateLimitFallback")
public LoanResponse applyForLoan(LoanRequest request) { ... }

resilience4j:
  ratelimiter:
    instances:
      loanApplication:
        limit-for-period: 5
        limit-refresh-period: 1s
        timeout-duration: 0s
```

Algorithm	How It Works
Token Bucket	Tokens refill at a fixed rate; each request consumes a token, request rejected if bucket is empty
Sliding Window	Counts requests in a rolling time window, rejects once the limit is exceeded
Fixed Window	Simple counter reset every fixed interval — can allow bursts right at window boundaries

22. Distributed Transactions & the Saga Pattern

22.1 Why Traditional Transactions Don't Work Across Services

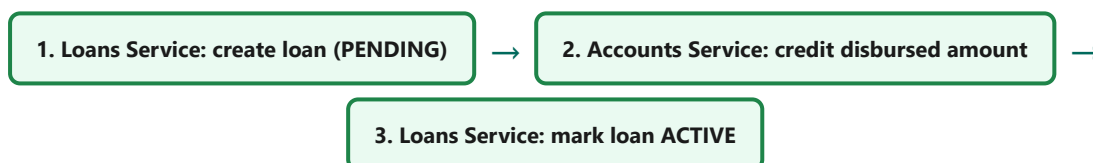
In a monolith, "create loan + deduct account balance" is one local ACID transaction. Across microservices with separate databases, there's no single transaction spanning both — a distributed 2-Phase-Commit (2PC) is possible but slow, fragile, and rarely used in modern microservices due to blocking/availability tradeoffs.

22.2 The Saga Pattern — Solution

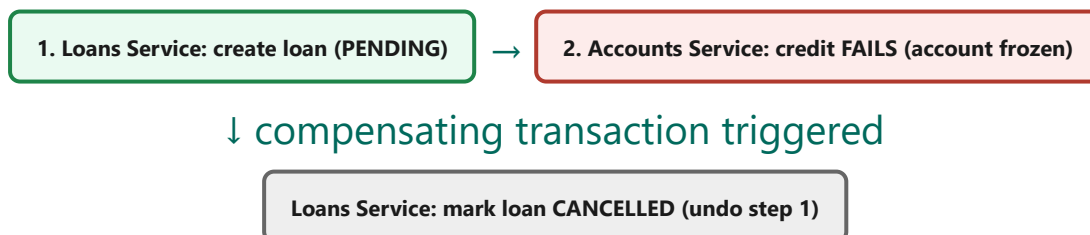
A Saga breaks a business transaction into a sequence of local transactions, each in one service, with a **compensating transaction** defined for each step to undo it if a later step fails — trading strict ACID consistency for eventual consistency with guaranteed rollback logic.

22.3 Real Example — "Apply for Loan and Disburse to Account" Saga

Loan Disbursement Saga — Happy Path



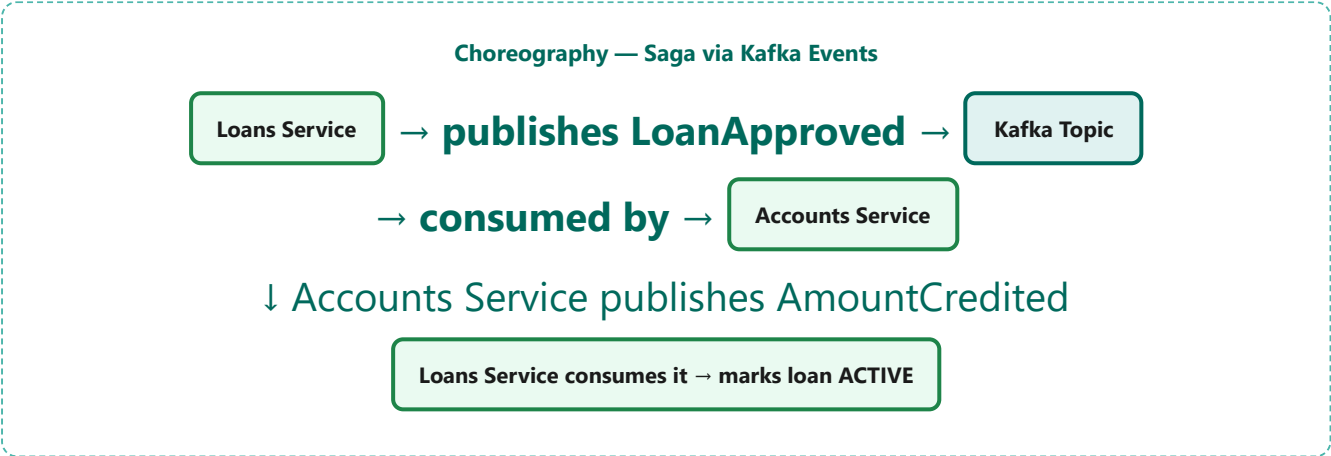
Loan Disbursement Saga — Failure & Compensation



23. Choreography vs Orchestration

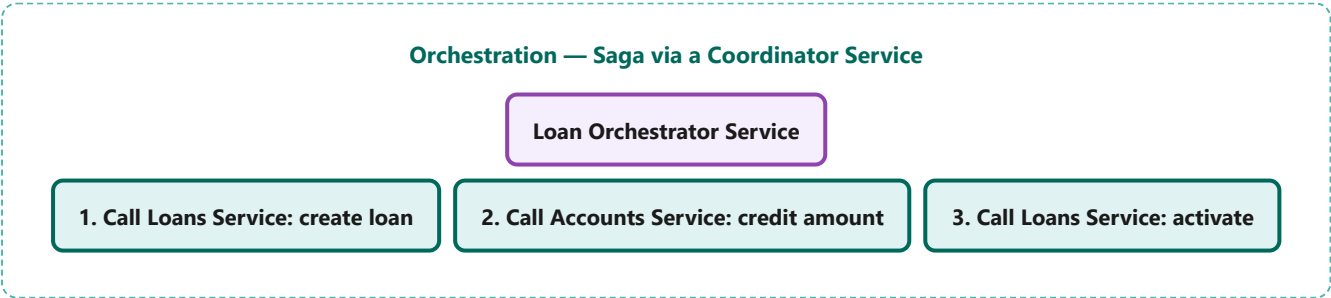
23.1 Choreography (Event-Driven, Decentralized)

Each service listens for events and reacts independently — no central coordinator. Loans Service publishes "LoanApproved", Accounts Service listens and credits the account, then publishes "AmountCredited", and so on.



23.2 Orchestration (Centralized Coordinator)

A dedicated orchestrator service explicitly calls each participant step by step and handles compensation logic centrally — easier to understand/monitor the overall flow, but introduces a central coordination point.



23.3 Choreography vs Orchestration — Comparison

Aspect	Choreography	Orchestration
Coordination	Decentralized — each service reacts to events	Centralized — one orchestrator drives the flow
Coupling	Loosely coupled, but flow is harder to see in one place	Coordinator is coupled to all participants, but flow logic is visible in one service
Complexity as steps grow	Gets harder to trace/debug ("who reacts to what" spreads across many services)	Scales better for complex flows — logic lives in one orchestrator
Best fit	Simple 2-3 step sagas, highly decoupled teams	Complex multi-step business processes (loan approval workflows with many steps)

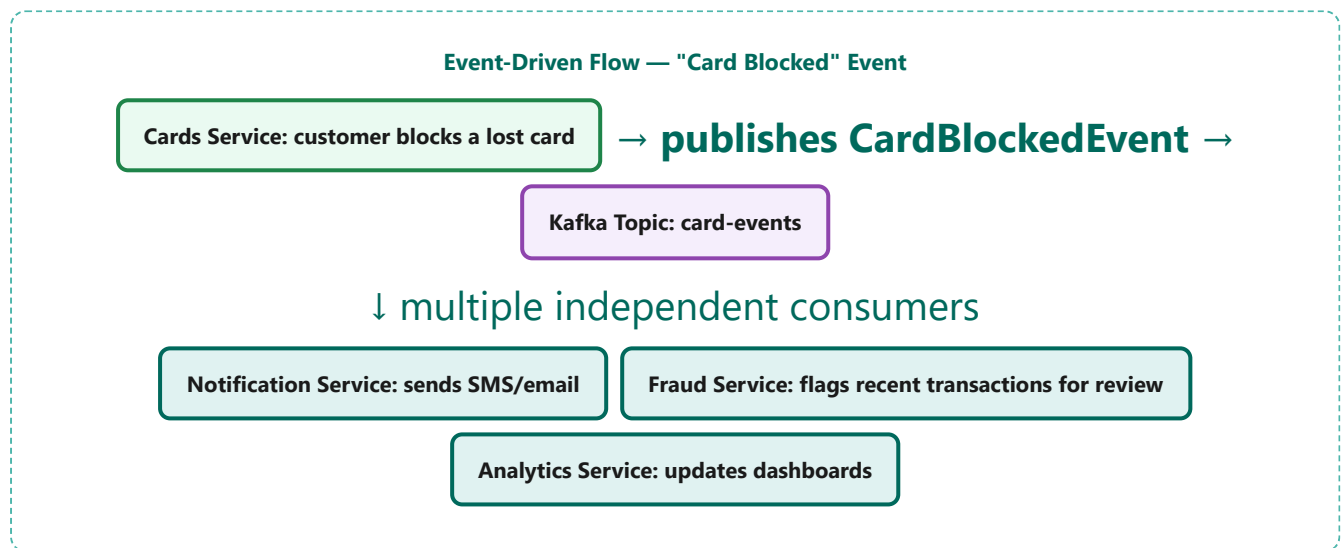
Common tech	Kafka/RabbitMQ events	Camunda, Spring State Machine, or a custom orchestrator service
-------------	-----------------------	---

24. Event-Driven Architecture

24.1 Core Idea

Instead of services calling each other directly and waiting for a response, services publish **events** ("something happened") to a message broker, and other interested services subscribe and react independently and asynchronously.

24.2 Real Event Flow — Banking Example



24.3 Benefits of Event-Driven Architecture

- Services are decoupled — Cards Service doesn't need to know Notification/Fraud/Analytics services exist at all.
- New consumers can be added later without changing the publisher.
- Naturally resilient to a consumer being temporarily down — messages queue up and are processed once it recovers.

Trade-off: consistency becomes **eventual**, not immediate — for a brief window, Fraud Service may not yet know about a just-blocked card. Design must account for this (Section 25 — Idempotency, and careful UX messaging like "your request is being processed").

25. Synchronous vs Asynchronous Communication

Aspect	Synchronous (REST/gRPC/Feign)	Asynchronous (Kafka/RabbitMQ)
Caller behavior	Waits for a response (blocking) or immediate reactive callback	Fire-and-forget — publishes and moves on
Coupling	Tighter — caller depends on callee being up and responsive right now	Looser — broker buffers messages if consumer is temporarily down
Use when	You need an immediate answer (e.g., "is this account valid?" before proceeding)	The caller doesn't need to wait (e.g., "send a notification," "update analytics")
Failure handling	Circuit breaker/retry/timeout patterns	Dead-letter queues, consumer retry, message replay
Real banking example	Gateway → Accounts Service (customer needs an immediate balance)	Cards Service → Notification Service (SMS can be sent a few seconds later)

Interview tip: A well-designed microservices system uses **both** — synchronous calls for read-your-own-write user-facing needs, asynchronous events for side effects and cross-service consistency that can tolerate a short delay.

26. Idempotency in Microservices

26.1 What Idempotency Means

An operation is idempotent if performing it multiple times produces the **same result** as performing it once — critical in distributed systems because network retries, timeouts, and duplicate message delivery are normal, expected occurrences, not edge cases.

26.2 Real Example — Idempotency Key for Loan Disbursement

```
@PostMapping("/loans/disburse")
public ResponseEntity<DisbursementResponse> disburse(
    @RequestHeader("Idempotency-Key") String idempotencyKey,
    @RequestBody DisbursementRequest request) {

    Optional<Disbursement> existing = disbursementRepository.findByIdempotencyKey(idempotencyKey);
    if (existing.isPresent()) {
        return ResponseEntity.ok(toResponse(existing.get())); // return the SAME result, don't re-disburse
    }

    Disbursement result = disbursementService.process(request, idempotencyKey);
    return ResponseEntity.ok(toResponse(result));
}
```

The client generates a unique **Idempotency-Key** per logical operation (e.g., a UUID stored client-side) and sends the exact same key on retries — the server recognizes the duplicate and returns the original result instead of processing it again.

26.3 Where Idempotency Matters Most

- Payment/disbursement operations (never double-charge or double-credit).
- Kafka consumers (a message can be delivered more than once — "at-least-once" delivery is the norm) — track processed message IDs to avoid double-processing.
- Retry logic on any state-changing (non-GET) API call.

27. Database per Service Pattern

27.1 The Rule

Each microservice owns its own database/schema exclusively — no other service is allowed to directly query or modify another service's tables. All access to a service's data goes through its API (or its published events).

Database per Service — Banking Example

Accounts Service
owns accounts_db

Cards Service
owns cards_db

Loans Service
owns loans_db

Loans Service **cannot** run `SELECT * FROM accounts_db.account` directly — it must call Accounts Service's API to get account details.

27.2 Why This Rule Exists

- Keeps services truly independently deployable — changing Accounts Service's schema doesn't break Loans Service, since Loans never touched those tables directly.
- Preserves clear ownership and encapsulation of business logic/validation within the owning service.
- Allows each service to pick the best-fit database technology (though most Java shops standardize on relational DBs for consistency).

27.3 The Cost — No More Cross-Service Joins

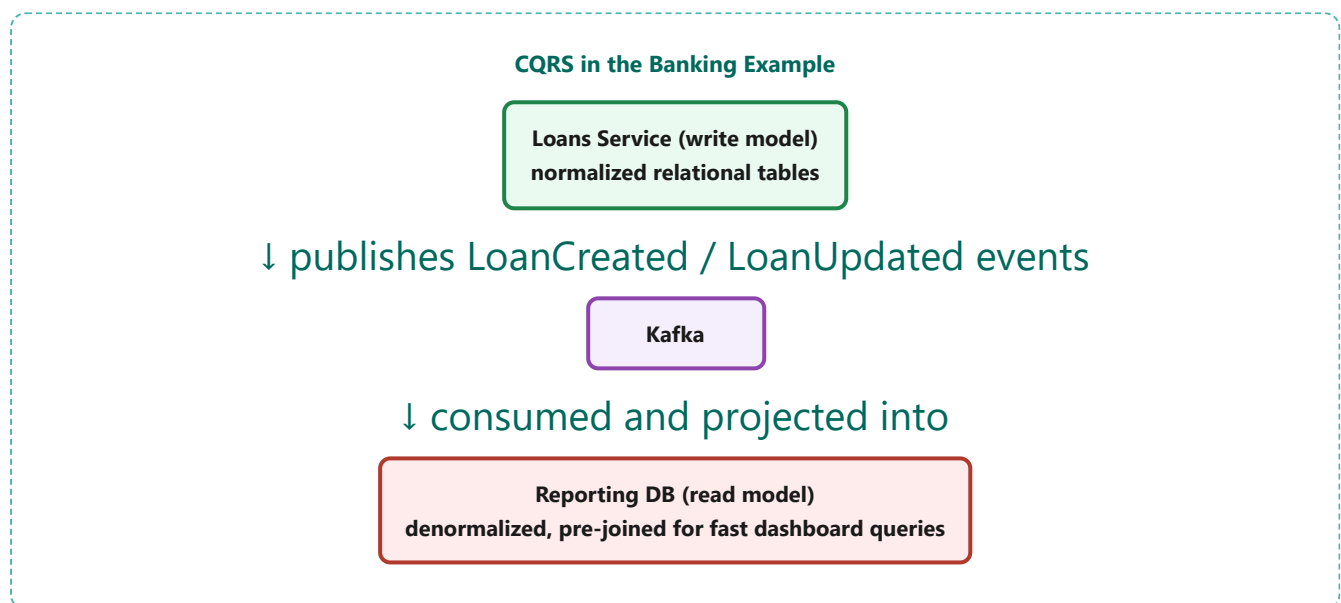
Building a report needing data from Accounts + Cards + Loans together can no longer be a simple SQL JOIN — it requires either API composition (call all three services and merge in code), a dedicated reporting/read database populated via events (see CQRS, Section 28), or a data warehouse fed by each service's data pipeline.

28. CQRS — Basic Understanding

28.1 What CQRS Means

CQRS (Command Query Responsibility Segregation) separates the model used to **write** data (Commands) from the model used to **read** data (Queries) — they can even live in entirely different databases, optimized independently.

28.2 Real Example — Loan Reporting Dashboard



28.3 Why Bother?

- Write side stays clean/normalized for correctness; read side is optimized purely for fast queries (denormalized, indexed for dashboards).
- Read load (dashboards, reports) never impacts the performance of the transactional write database.
- Complexity cost is real — you now maintain two models and an event pipeline keeping them in sync; only use CQRS where the read/write patterns genuinely differ enough to justify it (e.g., heavy reporting needs), not for simple CRUD services.

29. API Versioning Strategies

29.1 Why Versioning Matters

Once mobile apps and other services depend on your API, you can't just change its shape — you need a way to evolve the API without breaking existing consumers.

29.2 Common Strategies

Strategy	Example	Notes
URI Path Versioning	<code>/api/v1/accounts</code> , <code>/api/v2/accounts</code>	Most common, simple, visible in logs/URLs — used across most real orgs
Request Header Versioning	<code>Accept: application/vnd.bank.v2+json</code>	Cleaner URLs, but less discoverable/harder to test manually
Query Parameter Versioning	<code>/accounts?version=2</code>	Simple but easy to forget/misuse, less common in strict APIs

29.3 Real Spring Boot Example (URI Versioning)

```
@RestController
@RequestMapping("/api/v1/accounts")
public class AccountsControllerV1 { ... }

@RestController
@RequestMapping("/api/v2/accounts")
public class AccountsControllerV2 { ... } // new response shape, old v1 still supported for existing clients
```

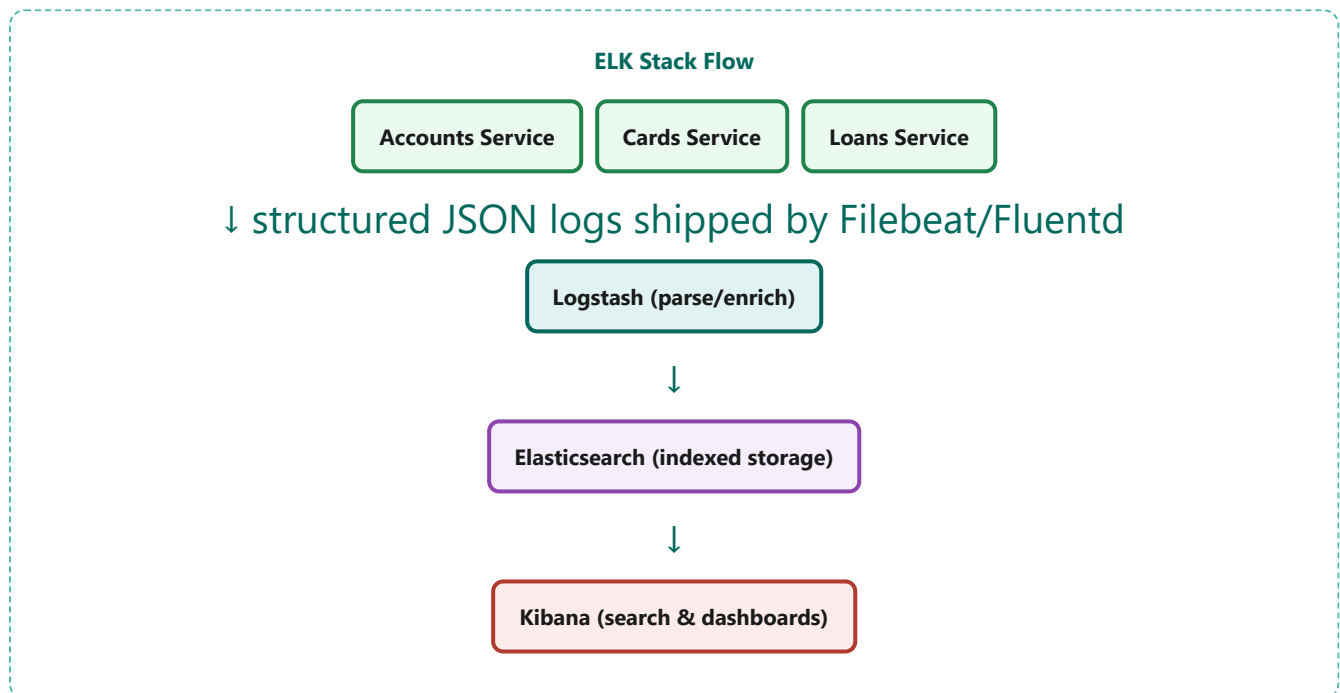
Real scenario: "When we added a new required field to the Accounts API response, we didn't change v1 at all — we shipped v2 in parallel, migrated the mobile app to v2 over a few sprints, then deprecated and eventually removed v1 six months later, with clear deprecation headers warning consumers ahead of time."

30. Centralized Logging — ELK Stack / Splunk

30.1 Why Centralized Logging Is Mandatory for Microservices

A single customer request might touch the Gateway, Accounts Service, Cards Service, and Kafka — with logs scattered across a dozen pods, manually SSHing into each one to grep logs is unworkable. Centralized logging aggregates every service's logs into one searchable place.

30.2 ELK Stack Architecture



30.3 Structured Logging in Spring Boot (Real Config)

```
// Use a correlation/trace ID in every log line so you can filter one request across all services
@Slf4j
@RestController
public class AccountsController {
    @GetMapping("/accounts/{id}")
    public AccountDto getAccount(@PathVariable Long id) {
        log.info("Fetching account id={} traceId={}", id, MDC.get("traceId"));
        ...
    }
}
```

logback-spring.xml pattern includes trace id from MDC (populated by tracing library, Section 31)

```
<pattern>%d{ISO8601} [%X{traceId}] %-5level %logger{36} - %msg%n</pattern>
```

30.4 Splunk as an Alternative

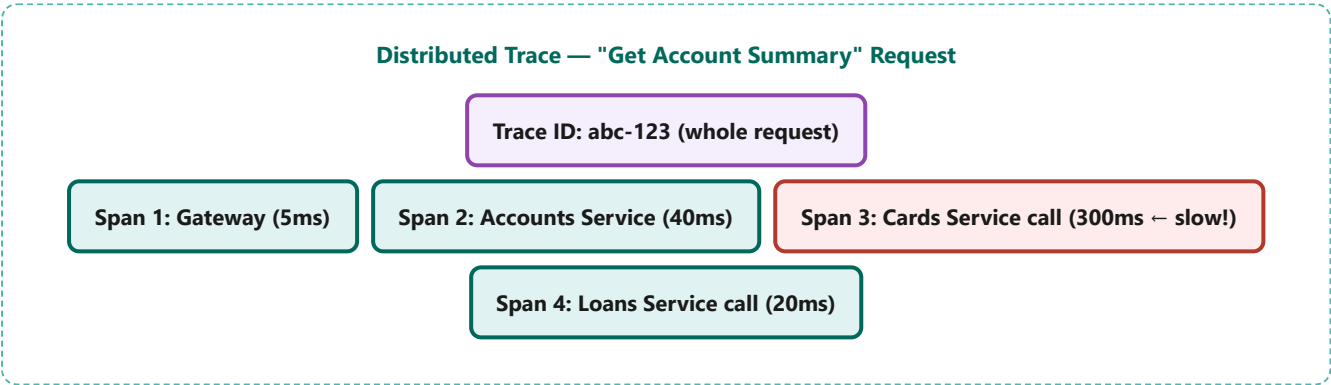
Splunk is a commercial log aggregation and analytics platform, often preferred by larger enterprises for its powerful search language (SPL), alerting, and compliance/audit features — functionally solves the same centralized logging problem as ELK but as a paid, fully managed/on-prem enterprise product.

31. Distributed Tracing — OpenTelemetry, Zipkin, Jaeger

31.1 The Problem Tracing Solves

Logs alone tell you what happened **inside** one service. Distributed tracing tells you the full journey of one request **across** all services it touched, with exact timing at each hop — essential for finding which service is the actual bottleneck in a slow multi-service call chain.

31.2 How It Works — Trace & Span



- **Trace:** represents the entire end-to-end request.
- **Span:** one unit of work within the trace (one service's handling of the request, or one downstream call) — spans are nested/linked to show the call hierarchy and exact duration.

31.3 OpenTelemetry — The Modern Standard

OpenTelemetry (OTel) is the current industry-standard, vendor-neutral instrumentation framework — it auto-instruments Spring Boot apps (HTTP calls, JDBC, Kafka) and can export trace data to **any** backend (Zipkin, Jaeger, Grafana Tempo, Datadog) without changing application code, avoiding vendor lock-in.

```
// build.gradle / pom.xml: add OpenTelemetry Java agent, attached at JVM startup, zero code changes needed
java -javaagent:opentelemetry-javaagent.jar \
  -Dotel.service.name=accounts-service \
  -Dotel.exporter.otlp.endpoint=http://otel-collector:4317 \
  -jar accounts-service.jar
```

31.4 Zipkin vs Jaeger

Tool	Origin	Notes
Zipkin	Twitter	Simpler, lightweight, easy to run locally, good for small-to-mid scale
Jaeger	Uber (CNCF project)	More scalable storage backends (Elasticsearch/Cassandra), richer UI, common in larger Kubernetes-native orgs

Real scenario: "Using Jaeger, we discovered that a 'slow account summary' complaint was actually caused by Cards Service making a synchronous, un-cached call to a third-party card network API on every request — the trace showed 300ms spent purely in that one span, immediately pointing us at the real bottleneck instead of guessing."

32. Monitoring & Metrics — Prometheus & Grafana

32.1 Metrics vs Logs vs Traces

Signal	Answers	Tooling
Logs	What exactly happened (detailed events, errors, stack traces)	ELK/Splunk
Metrics	How is the system performing overall, over time (aggregated numbers)	Prometheus/Grafana
Traces	Where did time go for one specific request across services	Zipkin/Jaeger/OTel

32.2 Spring Boot + Micrometer + Prometheus

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

```
# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health, info, prometheus, metrics
```

```
// custom business metric — e.g., count loan applications
@Service
@RequiredArgsConstructor
public class LoansService {
    private final MeterRegistry meterRegistry;

    public void applyForLoan(LoanRequest request) {
        meterRegistry.counter("loan.applications", "type", request.getLoanType()).increment();
        ...
    }
}
```

32.3 Grafana Dashboards (What Real Teams Watch)

- Request rate, error rate, and latency percentiles (p50/p95/p99) per service — the classic "RED metrics" (Rate, Errors, Duration).
- JVM metrics — heap usage, garbage collection pause times, thread pool saturation.
- Business metrics — loan applications per hour, card blocks per day, failed payment rate.
- Alertmanager rules firing to Slack/PagerDuty when error rate or latency crosses a threshold for a sustained period.

33. Service Registry, Health Checks & Spring Boot Actuator

33.1 Actuator — Built-In Operational Endpoints

Spring Boot Actuator exposes production-ready endpoints out of the box for health, metrics, and environment info — the backbone of Kubernetes probes, Eureka health checks, and monitoring dashboards.

Endpoint	Purpose
<code>/actuator/health</code>	Overall health status (UP/DOWN), including DB/Kafka connectivity checks
<code>/actuator/health/readiness</code>	Is the app ready to receive traffic (used by K8s readiness probes)
<code>/actuator/health/liveness</code>	Is the app alive at all (used by K8s liveness probes)
<code>/actuator/info</code>	Build/version metadata
<code>/actuator/metrics</code>	Raw metrics (also scraped via <code>/actuator/prometheus</code>)
<code>/actuator/env</code>	Current active configuration properties (should be secured in prod)
<code>/actuator/refresh</code>	Reloads <code>@RefreshScope</code> beans with new Config Server values

33.2 Custom Health Indicator

```
@Component
public class CardsServiceHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        boolean reachable = pingCardsService();
        return reachable ? Health.up().build()
            : Health.down().withDetail("reason", "Cards Service unreachable").build();
    }
}
```

33.3 How Eureka Uses Health Checks

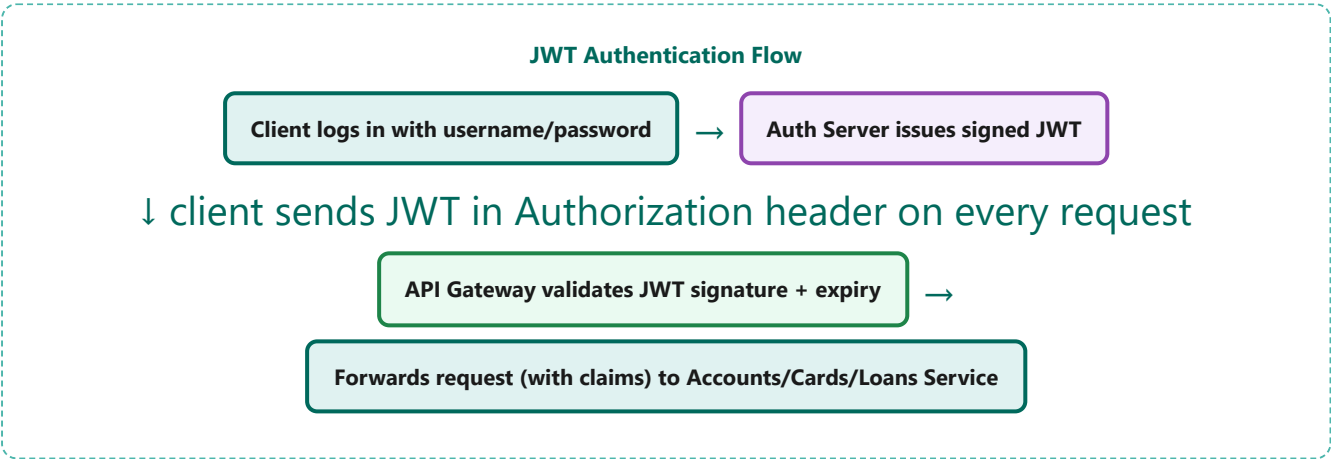
Eureka clients send heartbeats every 30 seconds by default; if a service's Actuator health check fails, Spring Cloud can be configured (`eureka.client.healthcheck.enabled=true`) to mark the instance OUT_OF_SERVICE in the registry, so it stops receiving new traffic without needing to be killed outright.

34. Security in Microservices — JWT, OAuth2, Keycloak

34.1 Why Security Is Harder in Microservices

In a monolith, authenticating once at the front door and using a server-side session is simple. Across many independently-scaled, stateless microservices, you need a token-based approach that any service can verify without a shared session store.

34.2 JWT (JSON Web Token) — How It Works



A JWT contains three parts — **Header** (algorithm), **Payload** (claims: user ID, roles, expiry), **Signature** (verifies it wasn't tampered with) — any service holding the public key/secret can verify it independently, with no need to call back to the auth server for every request.

34.3 OAuth2 — The Authorization Framework

OAuth2 Concept	Meaning
Resource Owner	The end user who owns the data
Client	The application requesting access (e.g., our React app)
Authorization Server	Issues tokens after authenticating the user (e.g., Keycloak)
Resource Server	The microservice that hosts the protected data and validates the token (e.g., Accounts Service)
Grant Types	Authorization Code (web apps), Client Credentials (service-to-service), Refresh Token (renew access without re-login)

34.4 Keycloak — Real Identity Provider

Keycloak is a popular open-source Identity and Access Management (IAM) server implementing OAuth2/OIDC — handles login screens, MFA, user federation (LDAP/AD), role/group management, and issues JWTs, so individual microservices never

need to implement login logic themselves.

```
# Spring Boot Resource Server config – validates JWTs issued by Keycloak
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://keycloak.mybank.com/realms/bank-realm

@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(auth -> auth
            .requestMatchers("/actuator/health").permitAll()
            .requestMatchers("/loans/**").hasRole("LOAN_OFFICER")
            .anyRequest().authenticated())
            .oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()));
        return http.build();
    }
}
```

34.5 Service-to-Service Security

Internal calls (Accounts → Cards via Feign) also carry the propagated JWT (or a service-specific client-credentials token) so downstream services can enforce their own authorization rules and know which original user/service initiated the request — never trust internal network location alone as "authenticated."

35. Caching — Redis

35.1 Why Cache in Microservices

Repeated calls for the same rarely-changing data (customer profile, card product catalog, interest rate tables) hitting the database or another service every time wastes resources and adds latency. Redis, an in-memory data store, is the most common caching layer used across Java microservices.

35.2 Spring Boot + Redis — Real Code

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>

@Service
public class CardsService {

    @Cacheable(value = "cardLimits", key = "#cardId")
    public CardLimitDto getCardLimit(Long cardId) {
        return cardLimitRepository.findByCardId(cardId); // only hits DB on cache miss
    }

    @CacheEvict(value = "cardLimits", key = "#cardId")
    public void updateCardLimit(Long cardId, BigDecimal newLimit) {
        cardLimitRepository.updateLimit(cardId, newLimit); // invalidate stale cache on update
    }
}

# application.yml
spring:
  cache:
    type: redis
  data:
    redis:
      host: redis-cluster
      port: 6379
```

35.3 What to Cache (and What Not To)

Good Candidates	Bad Candidates
Reference/lookup data (interest rates, product catalogs)	Real-time account balances (must always be fresh)
Expensive computed results (credit score calculations)	Highly volatile, security-sensitive data without careful TTL/invalidation
Session/token validation results (short TTL)	Data requiring strict read-your-own-write consistency

Real scenario: "Caching the card product catalog (which changes maybe once a week) in Redis with a 1-hour TTL cut Cards Service's database load by 70%, while account balance lookups deliberately bypass cache entirely since customers must always see their true current balance."

36. Message Brokers — Kafka & RabbitMQ

36.1 Apache Kafka — Real Producer/Consumer Code

```
// Producer — Cards Service publishes an event
@Service
@RequiredArgsConstructor
public class CardsEventPublisher {
    private final KafkaTemplate<String, CardBlockedEvent> kafkaTemplate;

    public void publishCardBlocked(CardBlockedEvent event) {
        kafkaTemplate.send("card-events", event.getCardId().toString(), event);
    }
}

// Consumer — Notification Service reacts
@Component
public class CardEventListener {
    @KafkaListener(topics = "card-events", groupId = "notification-service")
    public void handleCardBlocked(CardBlockedEvent event) {
        notificationService.sendSms(event.getCustomerPhone(), "Your card ending " + event.getLast4() + " has been blocked.");
    }
}
```

36.2 Kafka Core Concepts

Concept	Meaning
Topic	A named stream of events (e.g., <code>card-events</code>)
Partition	A topic is split into partitions for parallelism and ordering-per-key guarantees
Producer	Publishes events to a topic
Consumer Group	A set of consumers sharing the work of reading a topic — each partition read by only one consumer in the group
Offset	A consumer's position/checkpoint in a partition — enables replay and at-least-once delivery
Retention	Kafka retains messages for a configured period regardless of consumption — enables replay, unlike traditional queues

36.3 RabbitMQ — Basic Concepts (Alternative to Kafka)

RabbitMQ is a traditional message queue/broker (AMQP protocol) — simpler mental model (Exchange → Queue → Consumer), great for task queues and simpler pub/sub, but not built for the same massive-scale event streaming/replay use cases Kafka excels at.

```
@RabbitListener(queues = "loan.notification.queue")
public void handleLoanApproved(LoanApprovedEvent event) {
    notificationService.notifyCustomer(event.getCustomerId(), "Your loan has been approved!");
}
```

36.4 Kafka vs RabbitMQ — When to Use Which

Aspect	Kafka	RabbitMQ
Model	Distributed commit log — durable, replayable event stream	Traditional message queue/broker
Throughput	Very high, built for large-scale event streaming	Good for moderate throughput, more per-message routing flexibility
Message replay	Yes — consumers can rewind and reprocess historical events	No — once consumed/acked, typically gone
Best fit	Event-driven architecture, event sourcing, audit trails, analytics pipelines	Task queues, RPC-style messaging, simpler routing needs (fanout/topic exchanges)
Real banking example	Account/card/loan domain events feeding multiple downstream consumers + audit log	Sending a one-off email/SMS task to a background worker

37. Containerization (Docker) & Orchestration (Kubernetes)

Recap for Microservices

37.1 Why This Matters Specifically for Microservices

Microservices without containers/orchestration quickly become unmanageable — dozens of services, each needing consistent deployment, scaling, and networking, is exactly the problem Docker + Kubernetes were built to solve (full details in the companion Docker & Kubernetes guide).

37.2 One Deployment Per Microservice

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: cards-service, namespace: prod }
spec:
  replicas: 4
  selector: { matchLabels: { app: cards-service } }
  template:
    metadata: { labels: { app: cards-service } }
    spec:
      containers:
        - name: cards-service
          image: mybank.azurecr.io/cards-service:212
          ports: [{ containerPort: 8082 }]
          readinessProbe: { httpGet: { path: /actuator/health/readiness, port: 8082 } }
          livenessProbe: { httpGet: { path: /actuator/health/liveness, port: 8082 } }
---
apiVersion: v1
kind: Service
metadata: { name: cards-service, namespace: prod }
spec:
  selector: { app: cards-service }
  ports: [{ port: 80, targetPort: 8082 }]
```

37.3 Key Takeaways for Microservices Specifically

- Each microservice gets its own Deployment, Service, HPA — scaled and released completely independently.
- Namespaces isolate environments (dev/qa/prod), sometimes also per-team.
- Kubernetes' built-in DNS (`cards-service.prod.svc.cluster.local`) often complements or can replace Eureka on Kubernetes-native platforms — some organizations use both together during migration, others rely on Kubernetes Service discovery alone once fully containerized.

38. Deployment Strategies — Blue-Green, Canary, Rolling

38.1 Why This Matters More With Microservices

With many independently deployable services, each team needs a safe way to release their service frequently without risking the whole platform — these three strategies (fully detailed with diagrams in the companion CI/CD guide) are applied per-service.

38.2 Quick Recap Table

Strategy	How	Best Fit for Microservices
Rolling	Replace pods gradually, zero downtime	Default choice for most stateless microservices (Accounts, Cards, Loans)
Blue-Green	Two full environments, instant traffic switch	Critical services needing instant rollback (e.g., Loans disbursement engine)
Canary	Gradual traffic ramp-up to new version	High-traffic customer-facing services where blast radius must be minimized (Accounts Service)

Real scenario: "Cards Service (lower risk, high release frequency) uses Rolling updates by default. Loans Service (high financial risk per transaction) uses Canary — 5% of loan applications routed to the new version first, monitored closely for a full business day before ramping to 100%."

39. Fault Tolerance & High Availability

39.1 What "High Availability" Really Means

HA means the system keeps serving requests correctly even when individual components fail — no single point of failure, redundancy at every layer (multiple pod replicas, multiple availability zones, replicated databases).

39.2 Layers of Fault Tolerance in Our Banking Example

Layer	Fault Tolerance Mechanism
Pod/Instance level	Multiple replicas per service (min 3), Kubernetes self-healing restarts crashed pods
Node/Zone level	Pods spread across multiple Availability Zones via pod anti-affinity rules
Service-to-service calls	Circuit Breaker + Retry + Bulkhead + Timeout (Resilience4j)
Database level	Managed DB with automatic failover/replicas (e.g., Azure Database for MySQL with HA)
Messaging level	Kafka replication factor 3, consumer group rebalancing on consumer failure
Region level	Multi-region active-passive or active-active setup for disaster recovery of critical services

39.3 Timeout — The Most Underrated Resilience Pattern

```
resilience4j:
  timelimiter:
    instances:
      cardsService:
        timeout-duration: 2s    # never wait indefinitely on a downstream call
```

Without an explicit timeout, a hung downstream call can tie up caller threads indefinitely — always set a sane timeout on every inter-service call, tuned to realistic p99 latency plus margin.

40. How We Implement Microservices in Real-Time (End-to-End)

40.1 The Full Implementation Journey

1. **Domain analysis:** identify bounded contexts using Domain-Driven Design workshops with business stakeholders (Accounts, Cards, Loans emerge as natural boundaries).
2. **Define APIs first (API-first design):** write OpenAPI/Swagger contracts for each service before writing implementation code, so teams can work in parallel against agreed contracts.
3. **Scaffold infrastructure services:** stand up Eureka, Config Server, and API Gateway first — every business microservice depends on these.
4. **Build each microservice:** Spring Boot app with layered structure (Controller → Service → Repository), its own database, unit + integration tests (Testcontainers).
5. **Add resilience:** Resilience4j circuit breaker/retry/bulkhead/timeout on every inter-service call from day one, not as an afterthought.
6. **Add observability:** Actuator health endpoints, Micrometer metrics, OpenTelemetry tracing, structured JSON logging with correlation IDs — built in from the start.
7. **Add security:** JWT validation via Keycloak/OAuth2 at the Gateway and enforced again at each service (defense in depth).
8. **Containerize:** Dockerfile per service, docker-compose for local multi-service development.
9. **Set up CI/CD:** one pipeline per service (Azure DevOps/Jenkins/GitHub Actions) — build, test, scan, containerize, deploy independently.
10. **Deploy to Kubernetes:** namespaces per environment, HPA for autoscaling, Ingress for routing, monitored via Prometheus/Grafana.
11. **Iterate:** add new microservices only when a genuine new bounded context emerges — resist splitting too finely too early ("nanoservices" anti-pattern).

40.2 Real Team Structure (Conway's Law in Action)

"Organizations design systems that mirror their own communication structure" (Conway's Law). Our bank organizes around the same three services: an Accounts team, a Cards team, and a Loans team, each owning their service's full lifecycle — code, database, on-call, deployment — with a shared Platform/DevOps team owning Eureka, Gateway, Config Server, and the Kubernetes cluster itself.

40.3 Communication Between Teams (Not Just Services)

- **API contracts as the interface** between teams — changes to a shared API go through a review process (often using consumer-driven contract testing, e.g., Pact) so a breaking change is caught before it reaches production.
- **Shared platform standards** — common logging format, common base Docker images, common Resilience4j default configs — maintained by the Platform team, adopted by all service teams.
- **Regular architecture sync** — service teams meet periodically to discuss cross-service events/contracts, especially before introducing new Kafka topics or breaking API changes.

41. How to Maintain Microservices in Production (Real Organization Practices)

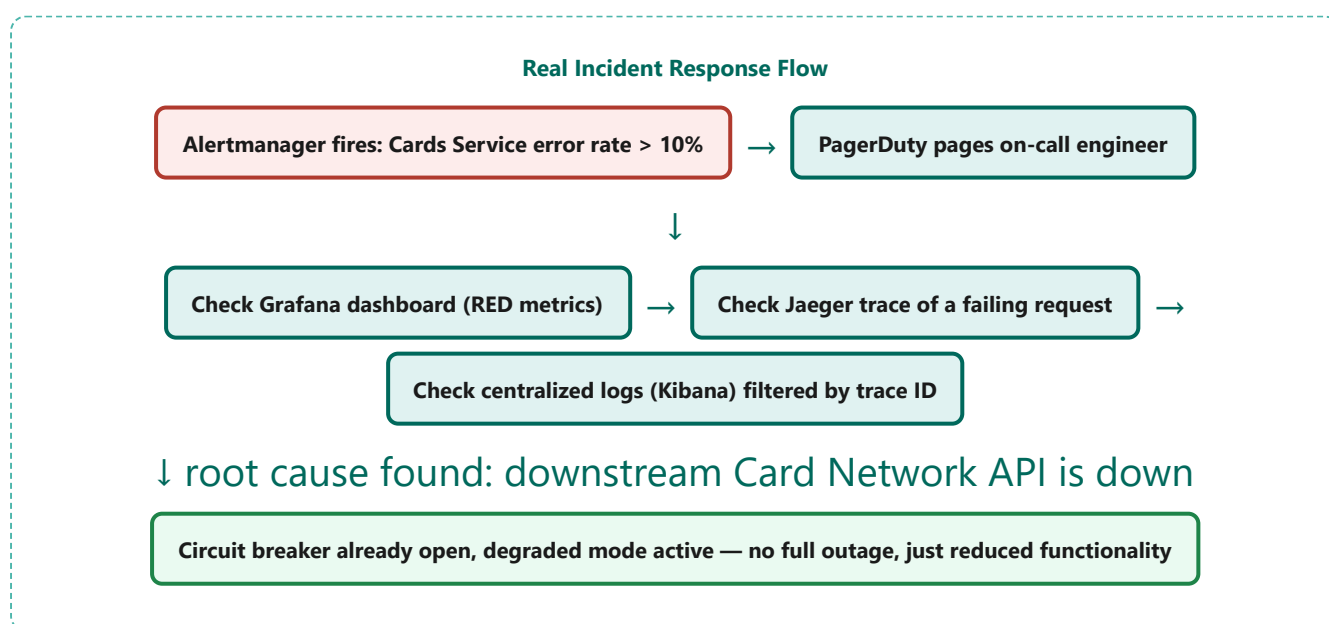
41.1 Day-2 Operations Checklist

- **On-call rotation per service team** — the team that built Cards Service is paged when Cards Service alerts fire ("you build it, you run it").
- **Runbooks** — documented step-by-step recovery procedures for common incidents (e.g., "Cards Service circuit breaker stuck open — check Card Network API status page first").
- **Dependency map kept up to date** — a living diagram of which services call which, so impact analysis during incidents is fast.
- **Regular chaos testing** — deliberately killing pods/injecting latency (Chaos Monkey-style) in non-prod to verify circuit breakers/retries actually work as designed.
- **Capacity planning** — reviewing HPA scaling history and resource requests/limits quarterly against real traffic growth.
- **Dependency upgrades** — regular scheduled Spring Boot/Spring Cloud version upgrades across all services (staggered, not all at once) to avoid falling far behind and facing a painful "big bang" upgrade later.

41.2 Versioning & Backward Compatibility Discipline

Since services deploy independently, Accounts Service v42 must be able to talk to Cards Service v38 (an older version, mid-rollout) without breaking — this means API changes must be backward compatible by default (add new optional fields, never remove/rename existing ones without a version bump per Section 29).

41.3 Real Incident Response Flow



41.4 Change Management for Independent Deployments

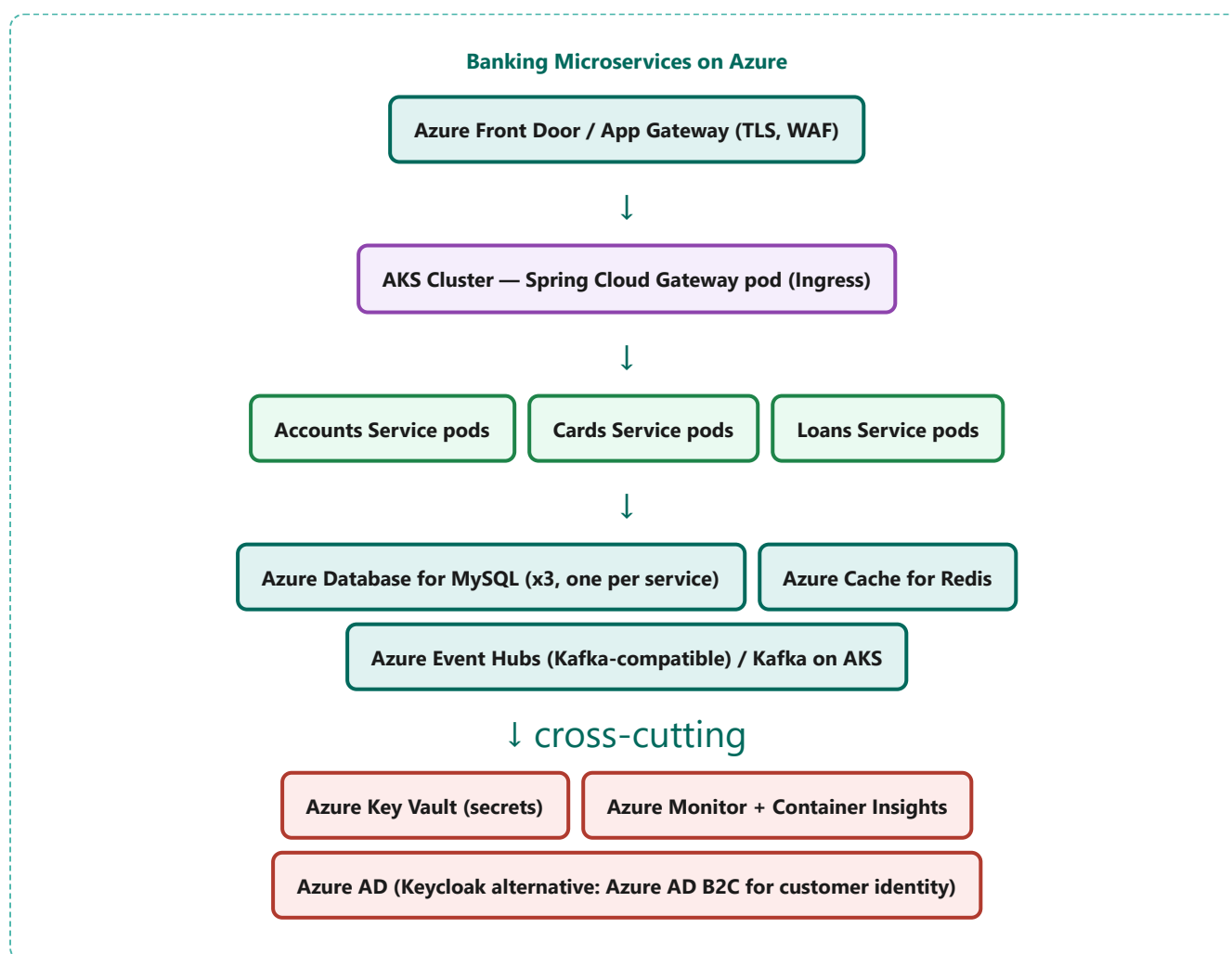
- Feature flags (via Config Server + `@RefreshScope`) to decouple deployment from release — deploy dark, enable gradually.
- Automated smoke tests post-deploy in every environment before traffic ramps up.
- Clear rollback procedure documented and tested for every service (image tag-based instant rollback via `kubectl rollout undo` or pipeline re-run).

42. Cloud Deployment — Azure Spring Apps, AKS and Azure Integration

42.1 Azure Options for Running Microservices

Azure Service	What It Provides	Best Fit
Azure Kubernetes Service (AKS)	Full Kubernetes control — you manage Deployments/Services/Ingress yourself	Teams wanting full container orchestration control, multi-cloud portability
Azure Spring Apps	Fully managed Spring Boot/Spring Cloud hosting — built-in Eureka-equivalent service registry and Config Server support, no Kubernetes YAML needed	Java/Spring Boot-heavy shops wanting less operational overhead than raw AKS
Azure Container Apps	Serverless containers built on Kubernetes/KEDA under the hood, simpler than AKS	Event-driven microservices, scale-to-zero workloads
Azure App Service	PaaS for web apps, simplest to deploy a single service to	Simpler monoliths or a handful of services, less microservices-native

42.2 Full Azure Architecture — Our Real Setup (AKS-Based)



42.3 Azure DevOps Pipeline Per Microservice (Real Pattern)

```
# azure-pipelines.yml — cards-service (one pipeline per service, same pattern repeated for all)
trigger: { branches: { include: [ main ] }, paths: { include: [ cards-service/* ] } }
pool: { vmImage: 'ubuntu-latest' }
variables: [ { group: cards-service-secrets } ]

stages:
- stage: Build
  jobs:
  - job: BuildTest
    steps:
    - task: Maven@4
      inputs: { mavenPomFile: 'cards-service/pom.xml', goals: 'clean verify' }
    - task: SonarQubePublish@5

- stage: DockerBuild
  jobs:
  - job: Docker
    steps:
    - task: Docker@2
      inputs: { containerRegistry: 'ACR-ServiceConnection', repository: 'cards-service', command:
'buildAndPush', tags: '$(Build.BuildId)' }

- stage: DeployAKS
  jobs:
  - deployment: Deploy
    environment: 'Production'
    strategy:
      runOnce:
        deploy:
          steps:
          - task: KubernetesManifest@1
            inputs:
              action: 'deploy'
              namespace: 'prod'
              manifests: 'cards-service/k8s/*.yaml'
              containers: 'mybank.azurecr.io/cards-service:$(Build.BuildId)'
```

Real scenario: "Each of our three services has its own Azure DevOps pipeline scoped by `paths.include`, so a change to Cards Service only triggers Cards Service's pipeline — Accounts and Loans are completely unaffected, exactly the independent-deployment benefit microservices are meant to deliver."

42.4 Cost & Scaling Notes

- Non-prod environments (Dev/QA) often share one smaller AKS cluster with separate namespaces to save cost; Production typically gets its own dedicated cluster.
- Azure Spring Apps can reduce operational cost for pure Spring Boot workloads by removing the need to manage Kubernetes YAML for service discovery/config directly.
- Reserved instances/savings plans for baseline AKS node capacity, autoscaling for burst capacity — a common real-world cost optimization.

43. Common Microservices Issues, Problems & Solutions

Issue	Cause	Real Solution
Cascading failures across services	No circuit breaker/timeout on inter-service calls	Add Resilience4j circuit breaker + timeout on every downstream call; design fallbacks
Data inconsistency between services	No saga/compensation logic for multi-service business transactions	Implement Saga pattern (choreography or orchestration) with explicit compensating transactions
Duplicate processing (double payment, duplicate notification)	Retries or at-least-once message delivery without idempotency handling	Idempotency keys on write APIs, track processed Kafka message IDs
"Which service is slow?" — can't tell	No distributed tracing in place	Instrument with OpenTelemetry, visualize in Jaeger/Zipkin
Debugging one request across 5 services is painful	Logs not correlated across services	Propagate a correlation/trace ID through every log line and every downstream call header
Config drift between environments	Manually managed application.yml per service per environment	Centralize with Spring Cloud Config Server backed by Git
Chatty services causing high latency	Over-decomposed services requiring many sequential network calls per user request	Reconsider service boundaries; consider API composition/BFF pattern, or merge overly fine-grained services
Database becomes a shared bottleneck	Multiple services accidentally sharing one database	Enforce Database per Service; migrate shared tables to the owning service via API
Difficult local development (need 10 services running)	No easy way to run the full stack locally	Docker Compose for full local stack; or use service virtualization/stubs (WireMock) for services not under active development
Breaking changes deployed without warning	No API versioning discipline or contract testing	Enforce backward-compatible changes + consumer-driven contract tests (Pact) in CI
Alert fatigue — too many noisy alerts	Alerting on raw metrics without meaningful thresholds/SLOs	Define SLOs per service, alert only on symptoms that affect users (error rate, latency), not every internal blip
Difficult to reason about system-wide impact of a change	No up-to-date service dependency map	Maintain (and auto-generate where possible, e.g., via tracing data) a live service dependency graph

44. Interview Questions & Answers (Basic → Advanced → Scenario Based)

Q1. What is the difference between monolithic and microservices architecture?

A monolith is one deployable unit with all modules sharing a codebase and database. Microservices split the application into small, independently deployable services, each with its own database, communicating over the network.

Q2. When would you recommend staying with a monolith instead of moving to microservices?

For small teams, early-stage products with unclear domain boundaries, low traffic internal tools, or organizations lacking mature CI/CD and monitoring practices — the operational overhead of microservices isn't justified until there's a proven need for independent scaling or release cadence.

Q3. How does Eureka enable service discovery, and what happens if Eureka itself goes down?

Services register with Eureka and send heartbeats; clients query Eureka for healthy instances of a service name. Eureka clients cache the last known registry locally, so brief Eureka downtime doesn't immediately break service-to-service calls — but new service instances can't register until Eureka recovers.

Q4. Explain the Saga pattern and why it's needed.

Since microservices can't share a single ACID transaction across databases, a Saga breaks a business transaction into a sequence of local transactions per service, each with a compensating transaction to undo it if a later step fails — trading strict consistency for eventual consistency with guaranteed rollback.

Real scenario: Loan disbursement — if crediting the account fails after the loan was created, a compensating transaction cancels the loan record instead of leaving the system in an inconsistent state.

Q5. Difference between Choreography and Orchestration sagas?

Choreography is decentralized — each service reacts to events independently, no central coordinator. Orchestration has a dedicated coordinator service explicitly directing each step. Choreography suits simple flows; orchestration scales better for complex multi-step business processes.

Q6. What problem does a Circuit Breaker solve, and how does it differ from a simple try-catch?

A Circuit Breaker prevents cascading failures by tracking failure rates over a sliding window and "opening" to fail fast (skip the real call entirely) once a threshold is breached, giving the downstream service time to recover — a try-catch alone still makes the call and waits for it to fail/timeout every single time.

Q7. Why is idempotency important, and how do you implement it?

Network retries and at-least-once message delivery mean the same operation can be triggered more than once; idempotency ensures repeating it produces the same result instead of duplicate side effects (e.g., double payment). Implemented via an idempotency key that the server checks before processing.

Q8. Why does each microservice need its own database?

It preserves independent deployability and encapsulation — if services shared a database, a schema change for one service could break another, defeating the purpose of splitting into microservices. Cross-service data needs go through APIs or events, not direct queries.

Q9. How do you handle reporting/analytics that need data from multiple services?

Either API composition (call each service and merge results in code, fine for small-scale needs), or CQRS-style read models fed by domain events published to Kafka and projected into a dedicated reporting database optimized for queries.

Q10. What's the difference between REST and gRPC for inter-service communication, and when would you choose gRPC?

REST/JSON is simple, human-readable, and universally supported but slower; gRPC uses binary Protobuf over HTTP/2, much faster, ideal for high-throughput internal service-to-service calls where performance matters more than human readability (e.g., a fraud-scoring service called on every transaction).

Q11. How do you secure microservices end to end?

Authenticate once at the edge (API Gateway validates JWT issued by an OAuth2/OIDC provider like Keycloak), propagate the token to downstream services, and enforce authorization again at each service (defense in depth) rather than trusting internal network location as implicit trust.

Q12. How do you debug a slow request that spans multiple microservices?

Use distributed tracing (OpenTelemetry + Jaeger/Zipkin) to see the exact time spent in each service/span for that specific trace ID, then cross-reference centralized logs (ELK) filtered by the same trace/correlation ID to see detailed context for the slow span.

Q13. What is the Strangler Fig pattern and why use it over a full rewrite?

It incrementally extracts pieces of a monolith into microservices one module at a time, routing traffic gradually via a gateway, while the monolith keeps running — avoiding the high risk of a big-bang rewrite where the business can't ship features for a long freeze period.

Q14. How would you design the "close account" use case across Accounts, Cards, and Loans services?

Accounts Service synchronously checks (via REST/Feign) for active cards/loans before allowing closure; once closed, it publishes an AccountClosedEvent to Kafka, which Cards Service and Loans Service consume asynchronously to perform their own side effects (deactivate cards, flag loan records) — combining synchronous validation with asynchronous eventual-consistency side effects.

Q15. What are the biggest challenges you've faced with microservices in production, and how did you solve them?

Example: cascading failures from a slow downstream Card Network API — solved by adding Resilience4j circuit breaker + bulkhead + timeout, so a slow dependency degrades gracefully (fallback response) instead of exhausting Accounts Service's own thread pool and causing a full outage.

Q16. How do you deploy 20+ microservices without them stepping on each other?

Each service has its own independent CI/CD pipeline (scoped to its own repo/folder), its own Kubernetes Deployment/Service/HPA, and its own database — deployments are fully decoupled. Shared infrastructure (Gateway, Config Server, Eureka) is owned by a platform team and changed more conservatively.

Q17. What is API versioning and why is it necessary in microservices?

Since services deploy independently, older and newer versions of a service coexist during rollout, and external consumers (mobile apps) can't always upgrade instantly — versioning (typically URI path-based, e.g., /api/v2/) lets you evolve APIs without breaking existing consumers.

Q18. Explain the Bulkhead pattern with a real example.

Bulkhead isolates resources (e.g., thread pools) per downstream dependency so a slow/failing one doesn't exhaust resources needed for unrelated calls — e.g., giving Loans Service calls their own dedicated thread pool so a slowdown there doesn't also block Cards Service calls sharing the same caller.

Q19. How does Kubernetes complement microservices architecture?

It provides per-service independent scaling (HPA), self-healing (restarts/reschedules failed pods), rolling/canary/blue-green deployment support, built-in DNS-based service discovery, and namespace-based environment isolation — solving most of the operational challenges that come with running many small services.

Q20. What would you consider before splitting a monolith into microservices?

Whether bounded contexts are actually well understood (DDD analysis), whether the team has mature CI/CD/monitoring/containerization practices, whether there's a genuine need for independent scaling/release cadence per module, and a realistic incremental migration plan (Strangler Fig) rather than a risky big-bang rewrite.

45. Final Summary & Cheat Sheet

Monolith: one deployable unit, one database, simple to start with, but scales and releases as a whole.

Microservices: independently deployable services organized around business capabilities, each with its own database, communicating over REST/gRPC/events — powerful but operationally complex.

Our banking example: Accounts, Cards, and Loans services, each owning their data, communicating synchronously (Feign/REST) for immediate needs and asynchronously (Kafka) for decoupled side effects, protected by circuit breakers/retries/bulkheads, secured with JWT/OAuth2/Keycloak, observed via Actuator/Micrometer/Prometheus/Grafana/OpenTelemetry/Jaeger/ELK, and deployed independently via Azure DevOps to AKS.

The golden rule: don't adopt microservices because it's trendy — adopt it when bounded contexts are clear, teams need independent release cadence, and your organization has the DevOps maturity (CI/CD, containerization, observability) to operate a distributed system reliably.

45.1 Quick Cheat Sheet

Term	One-Line Meaning
Bounded Context	A clear business domain boundary — the basis for defining a microservice
Eureka	Service registry — services register and discover each other by name
API Gateway	Single entry point handling routing, auth, rate limiting
Feign	Declarative REST client with Eureka + load balancing integration
Circuit Breaker	Fails fast once a dependency's failure rate crosses a threshold
Retry	Re-attempts transient failures, dangerous without idempotency
Bulkhead	Isolates resources per dependency so one failure doesn't starve others
Saga	Sequence of local transactions + compensations replacing distributed ACID transactions
Choreography	Decentralized event reactions, no coordinator
Orchestration	Centralized coordinator drives the multi-step flow
Idempotency Key	Ensures repeated requests don't cause duplicate side effects
Database per Service	Each service owns its data exclusively — no cross-service DB access
CQRS	Separate write and read models, often synced via events
Distributed Tracing	Follows one request's full journey across services (Jaeger/Zipkin)
Kafka	Durable, replayable event streaming platform for async communication
JWT/OAuth2/Keycloak	Token-based auth verified independently by each stateless service
Strangler Fig	Incrementally replace a monolith module by module, no big-bang rewrite

Use this guide alongside the Git/GitHub/GitLab/Bitbucket guide, the CI/CD guide, and the Docker & Kubernetes guide — together they cover the complete real-world Java full-stack developer journey: writing and versioning code, automating build/test/deploy, packaging into containers, orchestrating on Kubernetes, and designing/operating a resilient microservices architecture in production.